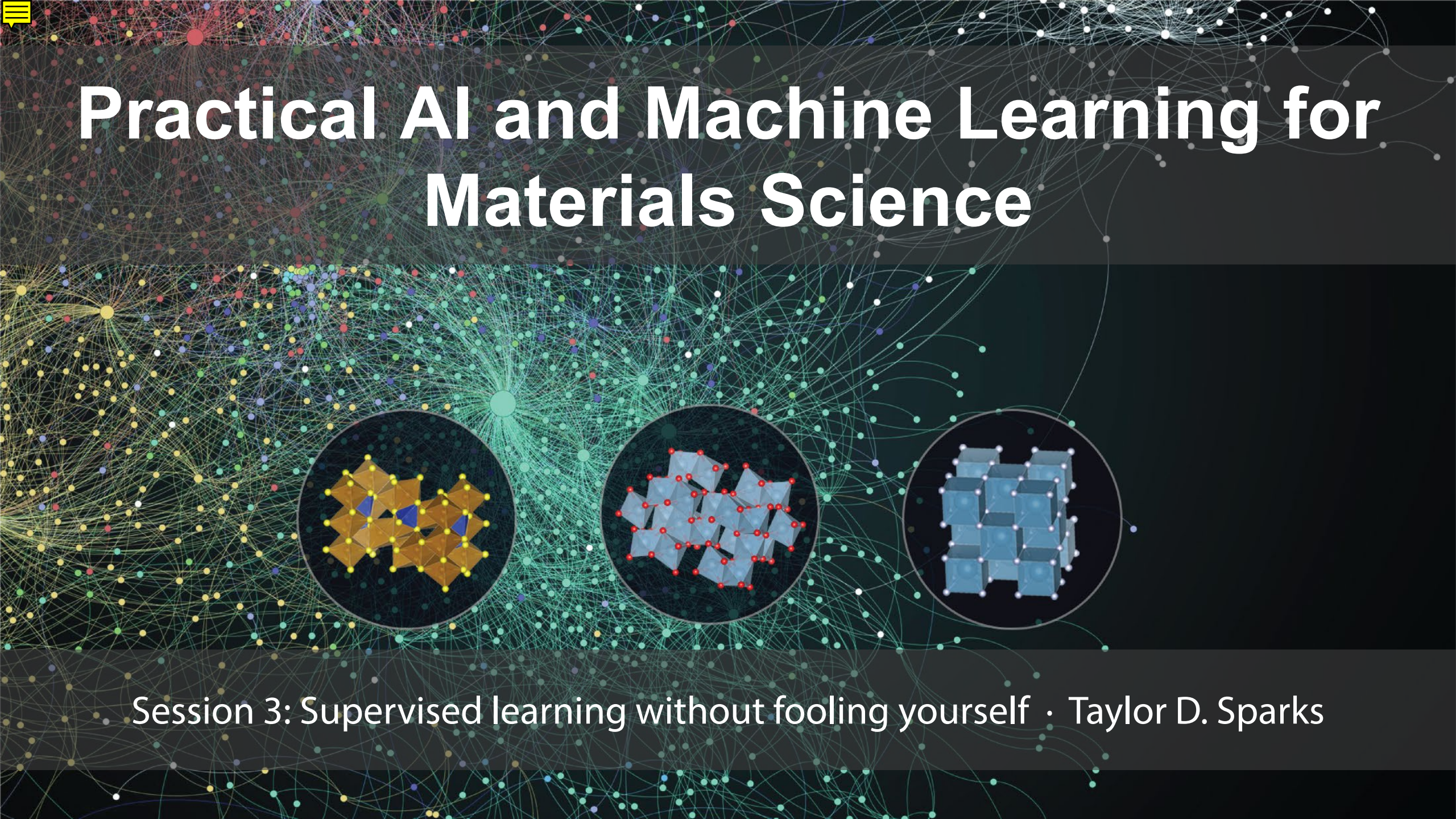
Practical AI and Machine Learning for Materials Science



Session 3: Supervised learning without fooling yourself · Taylor D. Sparks

Yesterday: four things to remember

1. Feature Representation matters

Often more than the choice of model

2. Chemistry helps most with little data

Domain features stand in for data you don't have

3. Always run a dumb baseline

Predict the mean, one-hot, random vectors

4. Know what you can't see

Polymorphs, microstructure, processing

Today: is my model actually any good, and how would I know?

You told us what you want; here is where each one lands

Thermoelectrics and energy materials

Today's challenge: power factor, p vs n, zT

Today + S5

Process/formulation optimization

Sintering, MOCVD, casting, coatings, concrete

Session 5

Micrographs

Segmentation, TEM, EBSD, grain and pore size

Session 4

Physics + ML

Physics-informed models, CFD / DFT / MD data

Sessions 4-5

LLMs and new tools

Literature mining, local vs cloud, foundation models

Session 6

Getting started, in industry

Choosing methods with confidence; from lab to plant

Session 6

Keep the questions coming: chat, email or GitHub issues

Your logistics questions, answered

Will the materials stay available?	Yes: the GitHub repository stays public after the course
Where are the slides?	slides/ folder, as PDFs with speaker notes, after each session
How do I keep my notebook edits?	In Colab: File → Save a copy in Drive, before you edit
How were the notebooks made, and checked?	Claude Code, then run end to end and checked: we'll do examples of Claude Code and Codex in Session 6

Not long ago, a materials ML model took about 14 lines of Python

```
1  import pandas as pd
2  from CBFV import composition
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.ensemble import RandomForestRegressor
6  from sklearn.metrics import mean_absolute_error
7
8  df = pd.read_csv("band_gap.csv")
9  X, y, formulae, skipped = composition.generate_features(df)
10 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)
11 scaler = StandardScaler().fit(X_tr)
12 model = RandomForestRegressor().fit(scaler.transform(X_tr), y_tr)
13 y_pred = model.predict(scaler.transform(X_te))
14 print(mean_absolute_error(y_te, y_pred))
```

load
featurize
split
scale
fit
predict
score

Load, featurize, split, scale, fit, predict: the whole Session 2 workflow fits on one screen

Today, one sentence in Claude Code writes the whole notebook

CLAUDE CODE • VS Code

> Make a Jupyter notebook that predicts band gap from band_gap.csv. Featurize the formulas with CBFV, split train/test, scale, fit a random forest, report MAE and plot predicted vs. actual.

- Created band_gap_model.ipynb (6 cells)
- Ran all cells: MAE 0.42 eV, parity plot saved

No errors. Looks great?

What nobody asked:

Are the same formulas in train and test?

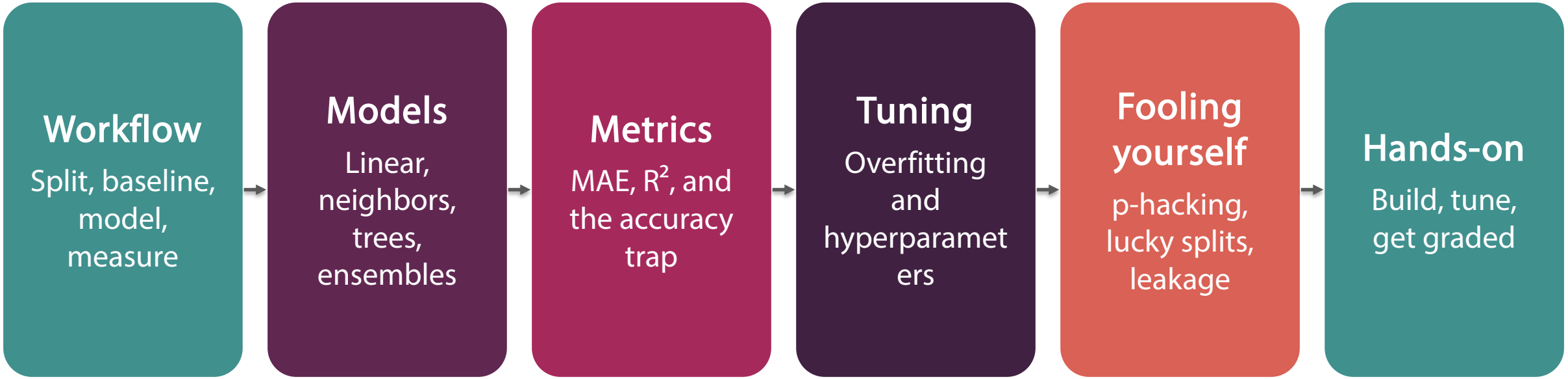
Is a random split the question I care about?

Is 0.42 eV better than predicting the mean?

How many models did I try before this one?

Predicting is easier than ever, so understanding how ML goes wrong is now the essential skill

Today we build models and learn to be skeptical of ML

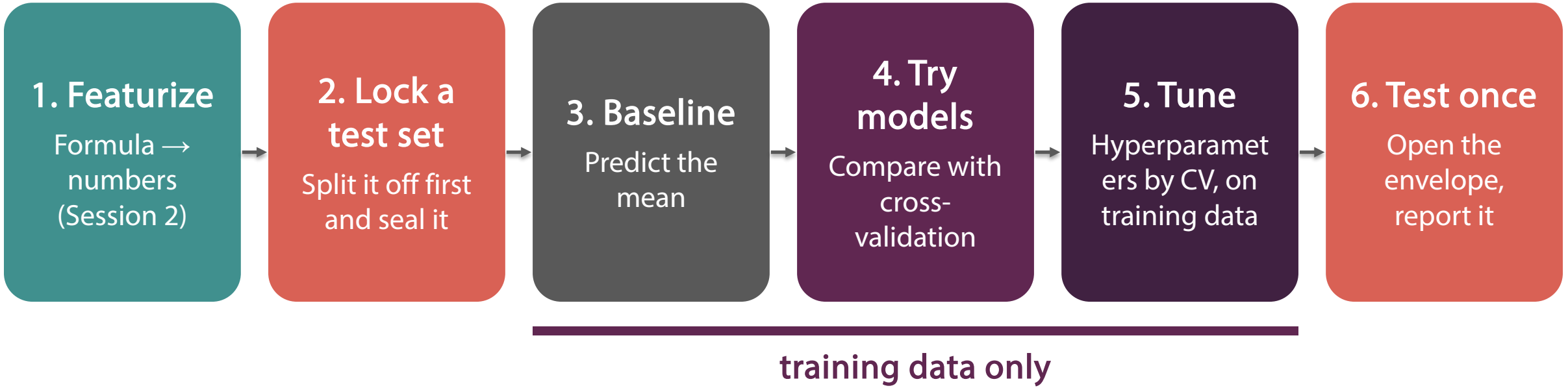


Key lesson: how you validate matters more than the accuracy number

The supervised learning workflow

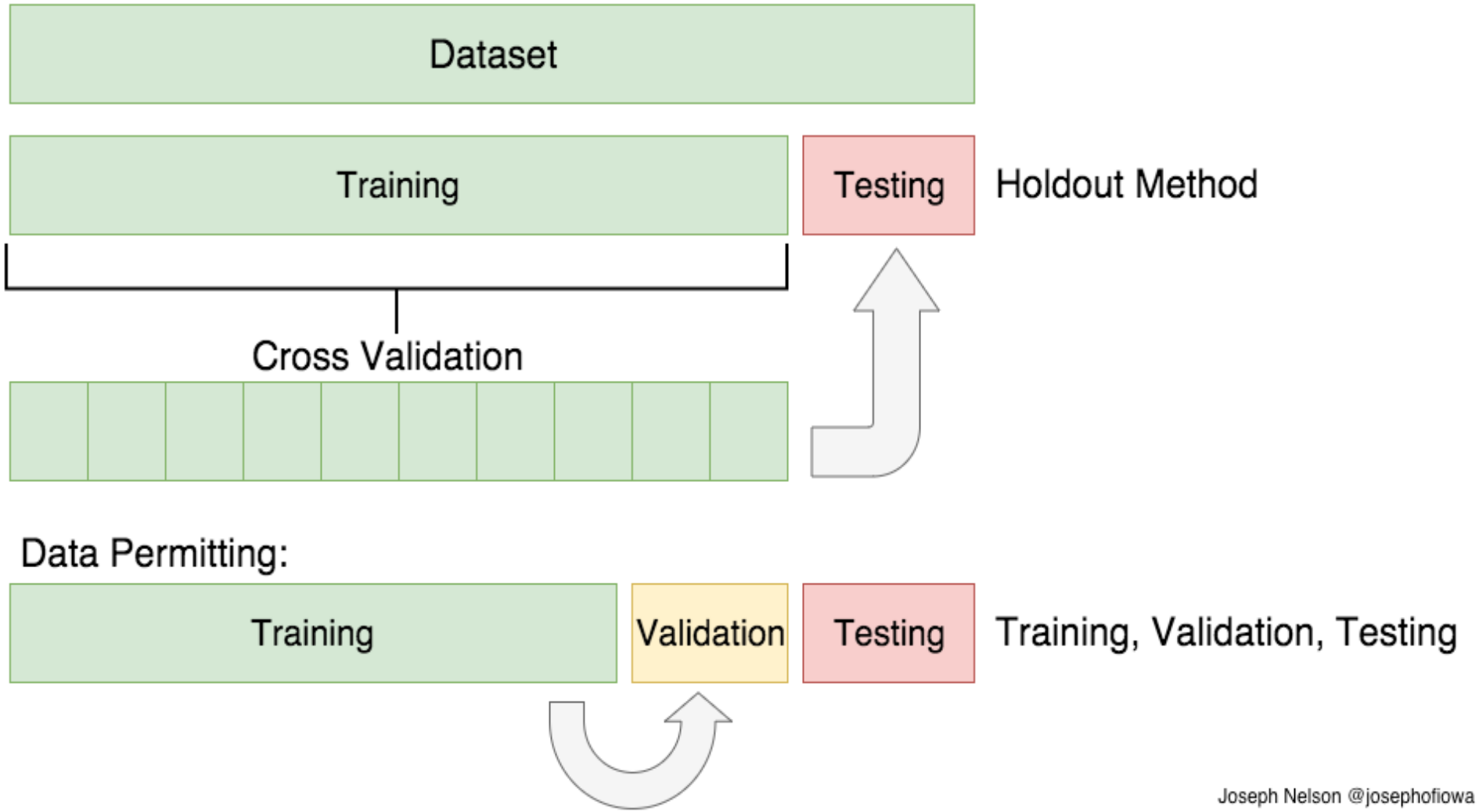
Split first, beat a baseline, measure honestly

Every supervised model follows the same six steps



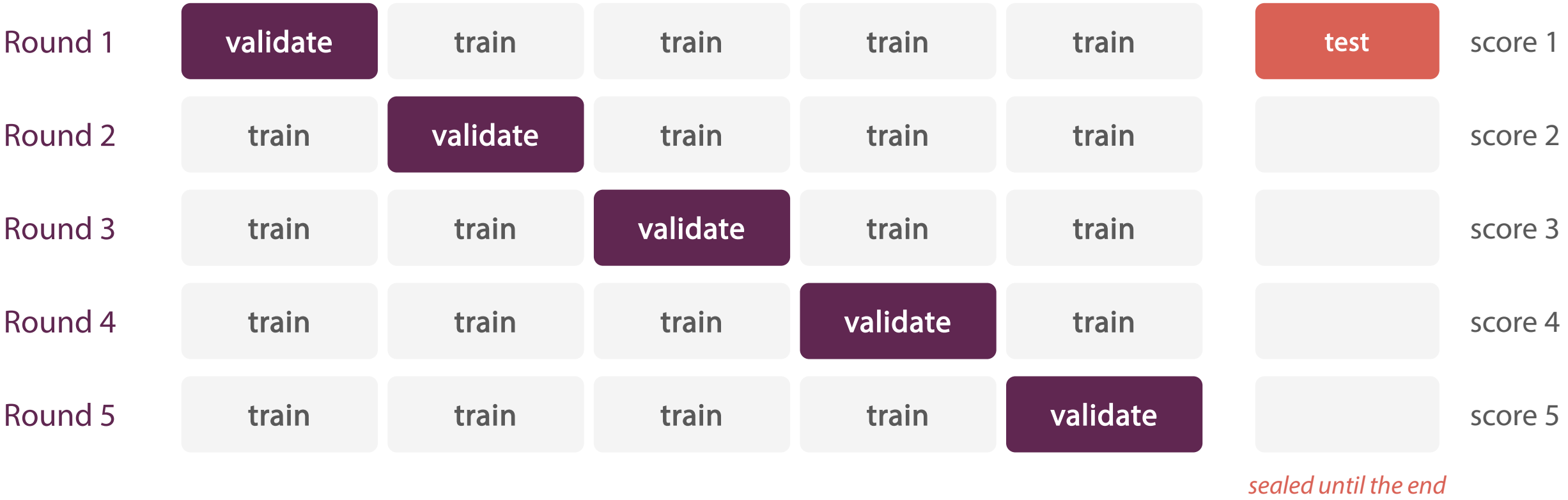
The test set answers one question: how will this do on materials I haven't made yet?

Training, validation and test sets each have one job



K-fold cross-validation gives every compound a turn in validation

Five scores instead of one: report the mean and the spread

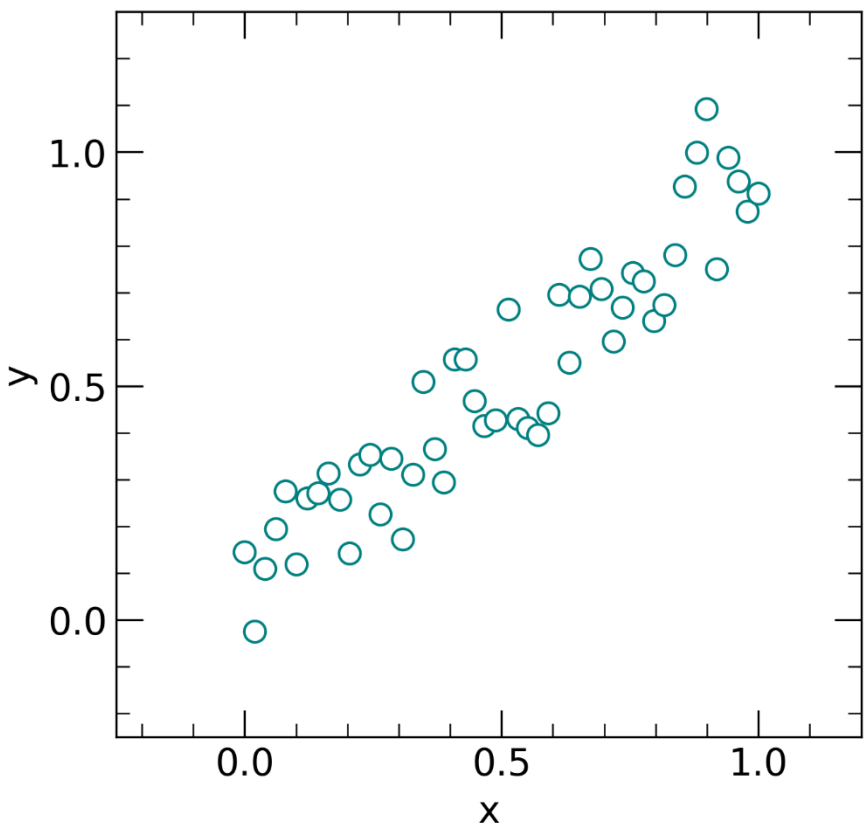


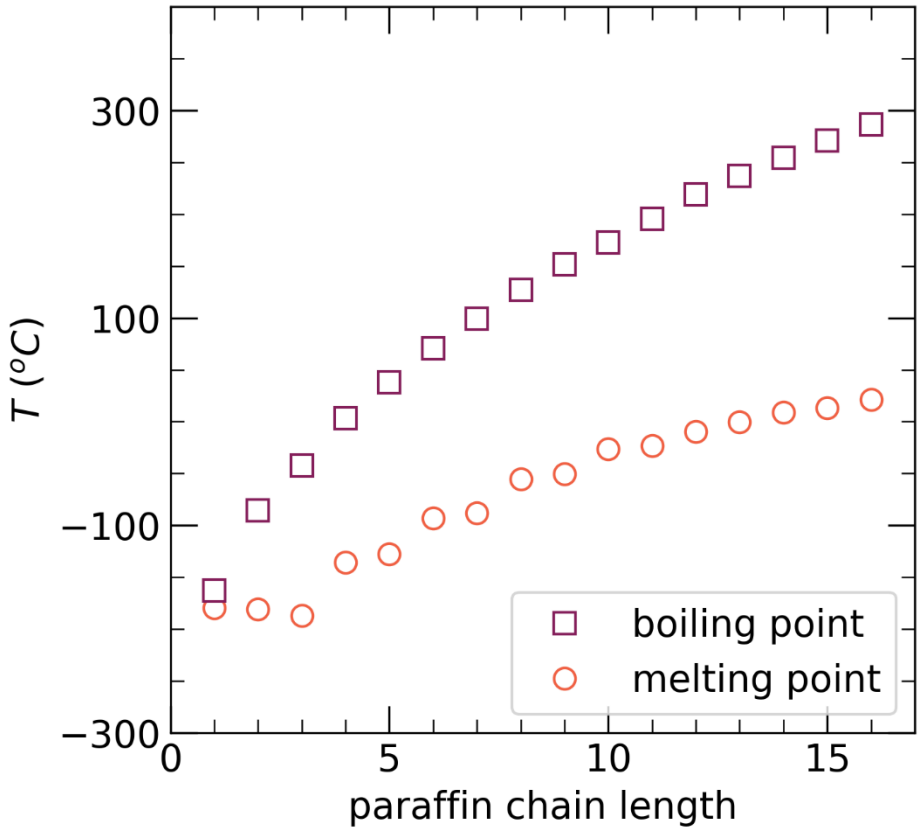
CV score = mean of five ± their standard deviation

Choosing a model

Linear models, neighbors, trees and ensembles

Both linear and non-linear models can fit curvature in data





Linear models must follow a strict format

$$dependent\ variable = constant + parameter * IV + parameter * IV + \cdots$$

$$y = mx + b$$

$$\mathrm{x_{final}} = x_{initial} + v_0 t + 1/2 a t^2$$

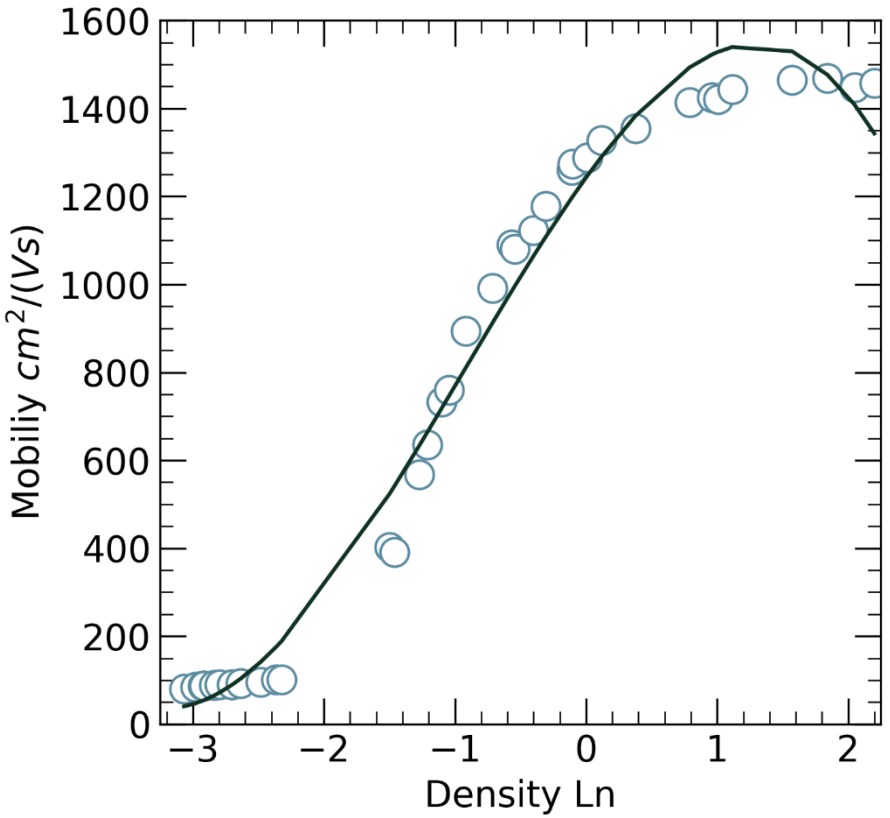
Linear models consist of sums of parameters multiplied by independent variables (IV)

Nonlinear models are anything that don't follow this form

Linear models are easy to fit and read; non-linear ones fit more

Benefits of linear models	Benefits of nonlinear models
Easy to implement	Harder to implement
Easy to interpret	Harder to interpret
Easily obtained statistics to assess model	Can fit more complicated trends in data (but this can be hard to accomplish)
	No R^2, no p-values possible for the parameter estimates

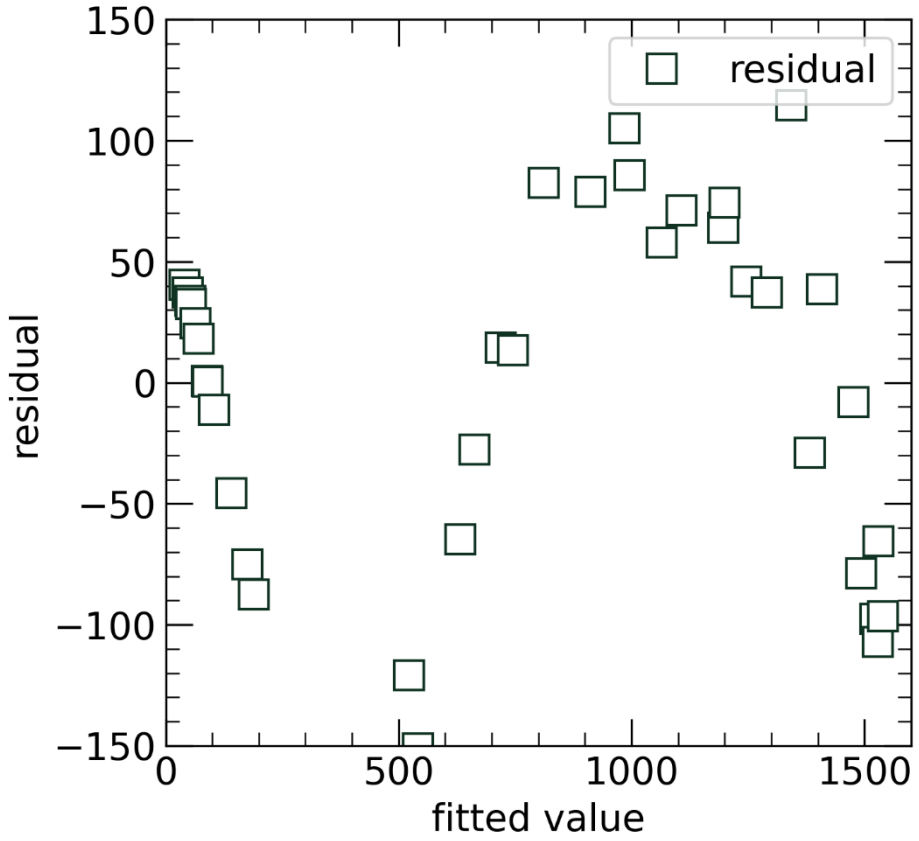
R^2 = 98.5% looks great, but the fit is systematically wrong



R^2 of 98.5% so, pretty good… right?

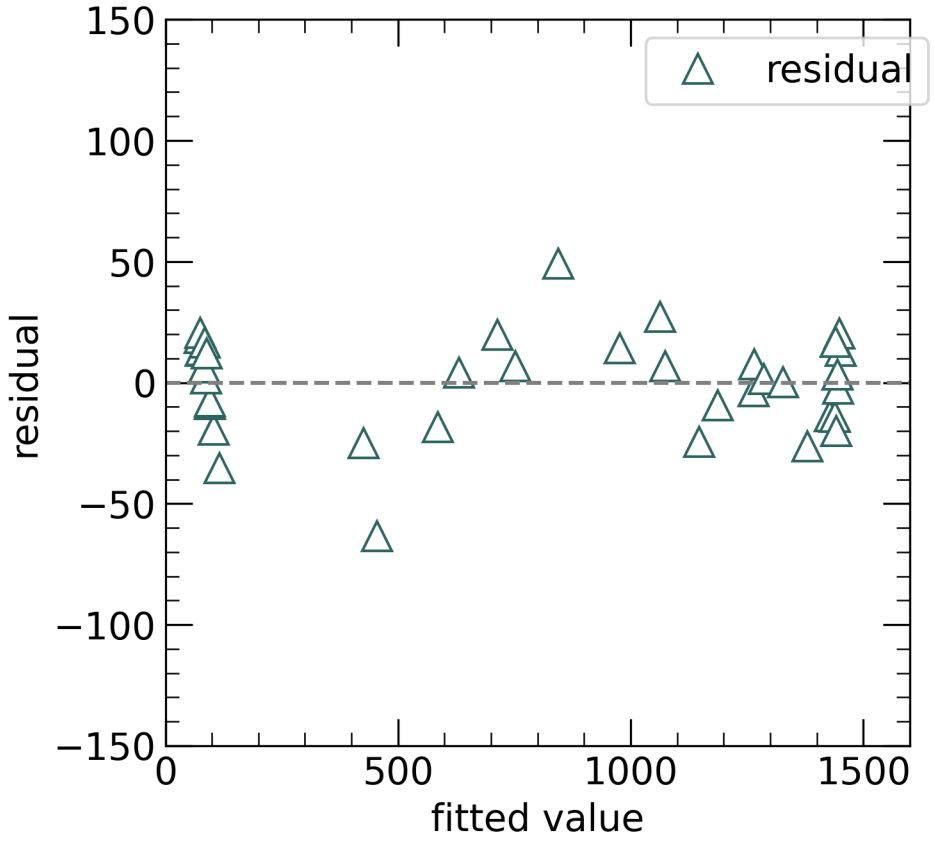
Clearly it overestimates and underestimates portions of the data in systematic (biased) ways

Always plot the residuals: patterns mean the model is biased



A model that is systematically incorrect is biased (R^2 doesn't tell us the whole story!)

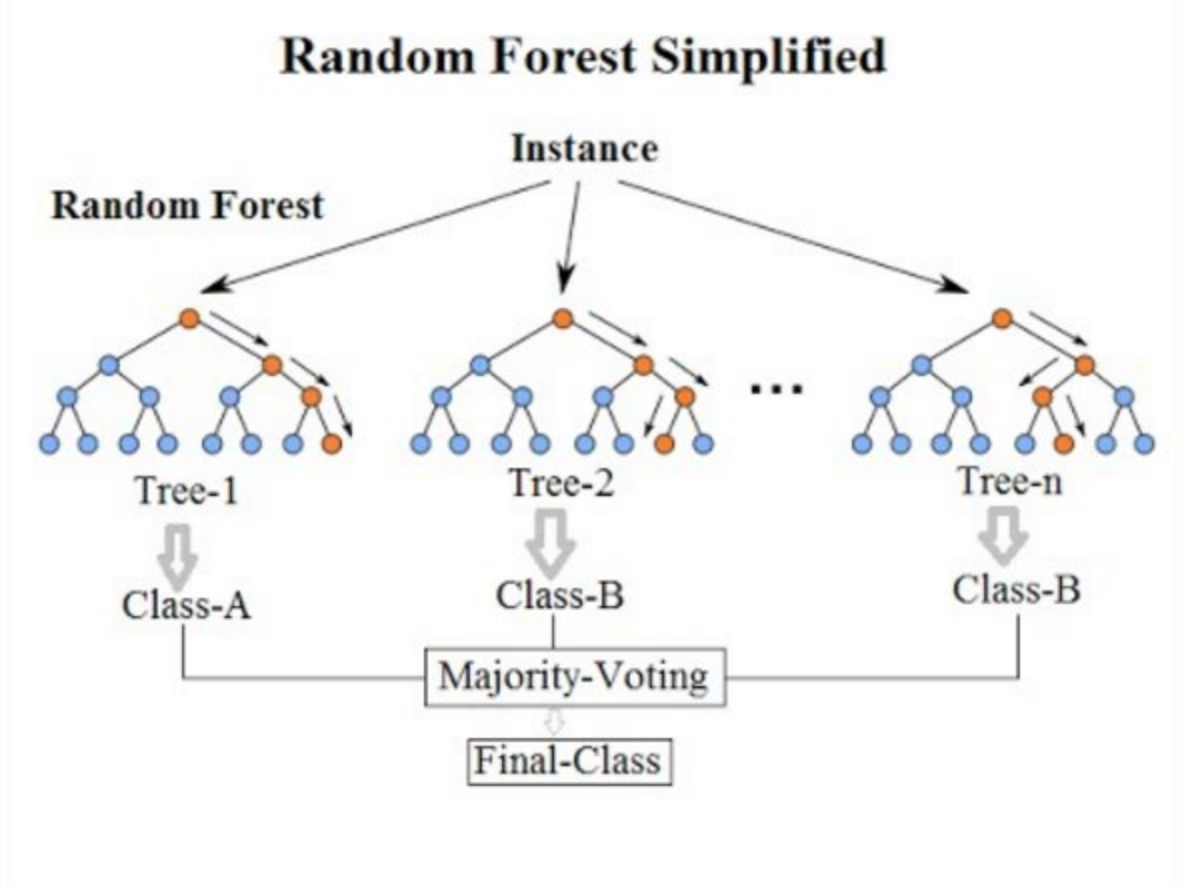
A better-shaped model leaves residuals that look like noise



The residual from the rational function is improved and less biased

Curve fit parameters determined by Scipy.optimize curve_fit() method

A random forest averages hundreds of slightly different trees



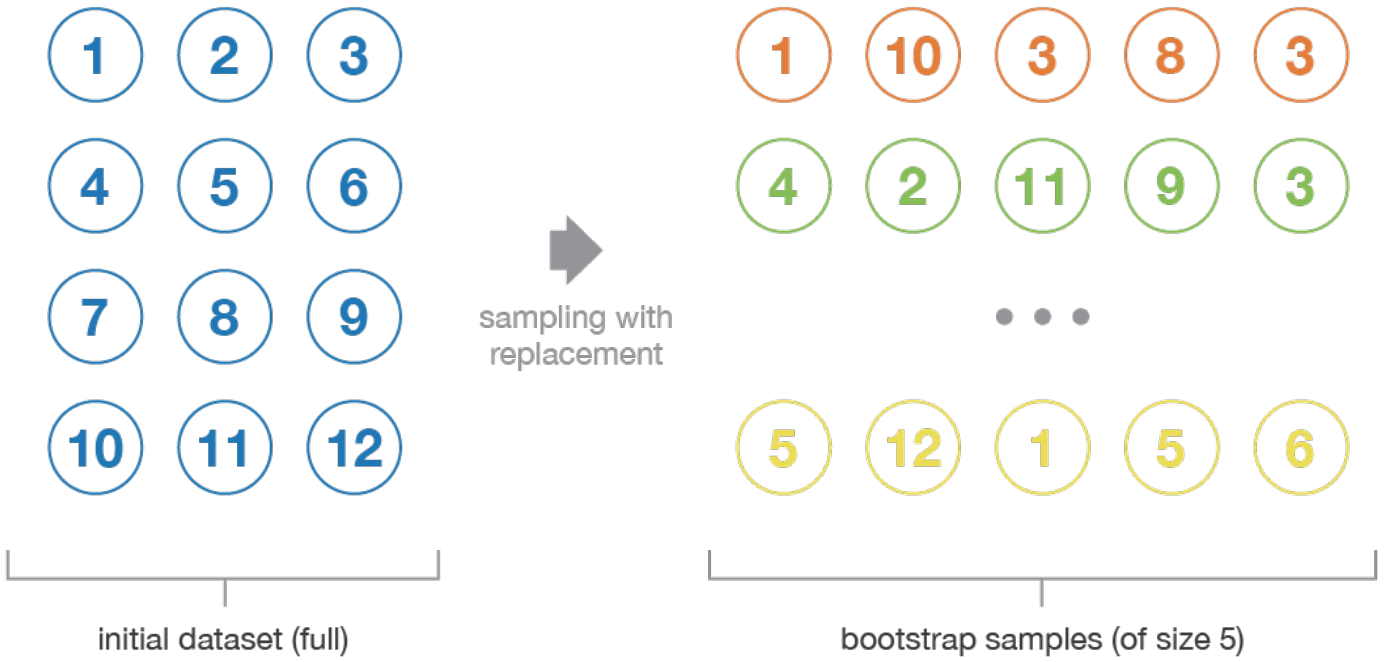
Multiple small trees can be calculated in parallel*

Different trees can sample different features

*depending on algorithm!

https://en.wikipedia.org/wiki/Random_forest#/media/File:Random_forest_diagram_complete.png

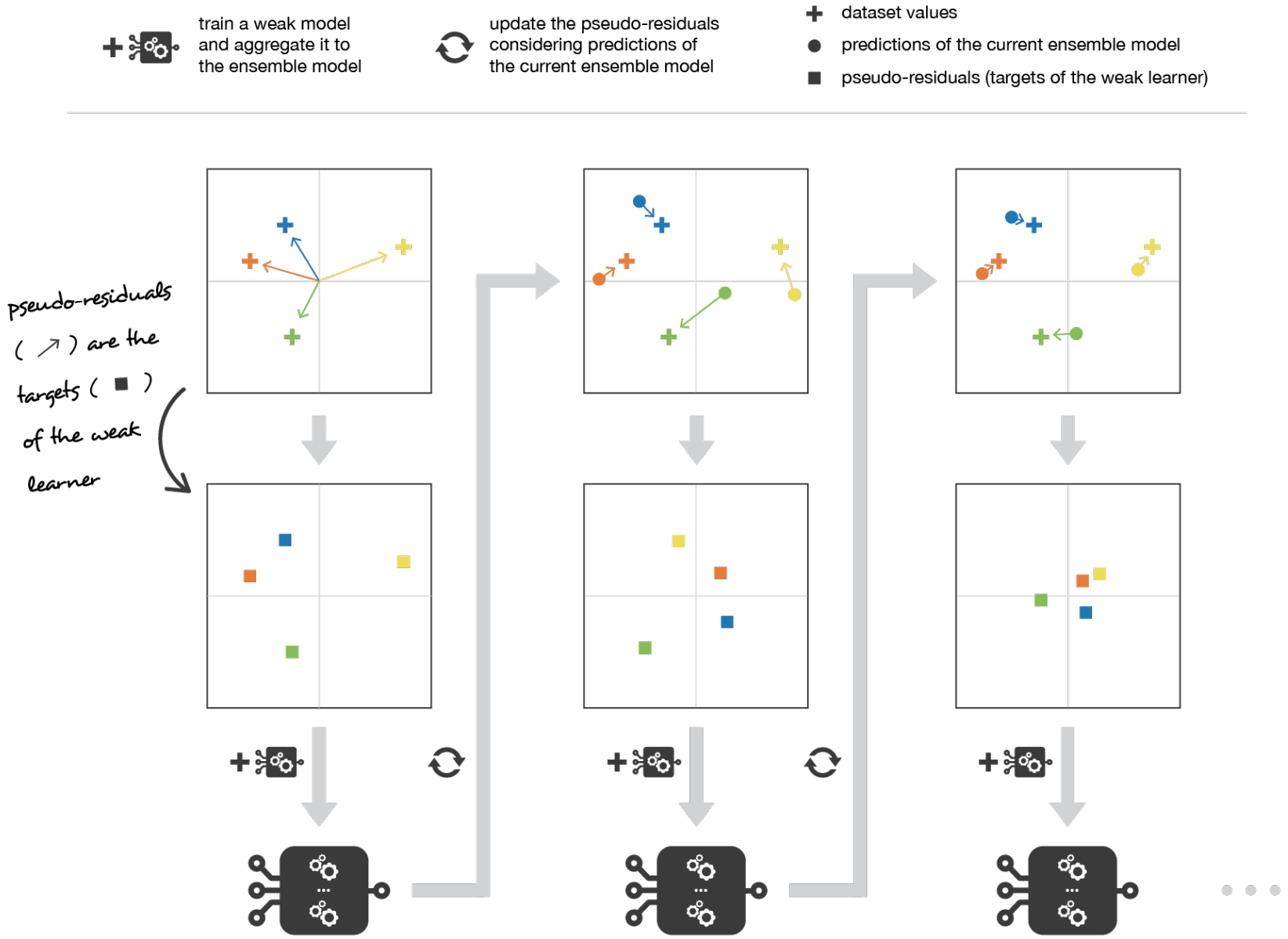
Bagging: train each tree on a resampled copy of the data



Bootstrapping:
generating samples of size B (called bootstrap samples) from an initial dataset of size N by randomly drawing with replacement B observations.

https://towardsdatascience.com/ensemble-methods-bagging-boosting-and-stacking-c9214a10a205

Boosting: each new tree corrects the previous trees' errors



A short list covers most materials problems

Dummy
Predict the mean. The bar to clear

Ridge / Lasso
Linear + penalty. Fast, readable. Scale first

k-nearest neighbors
Average similar compounds. Scale first

Random forest
Robust default, little tuning

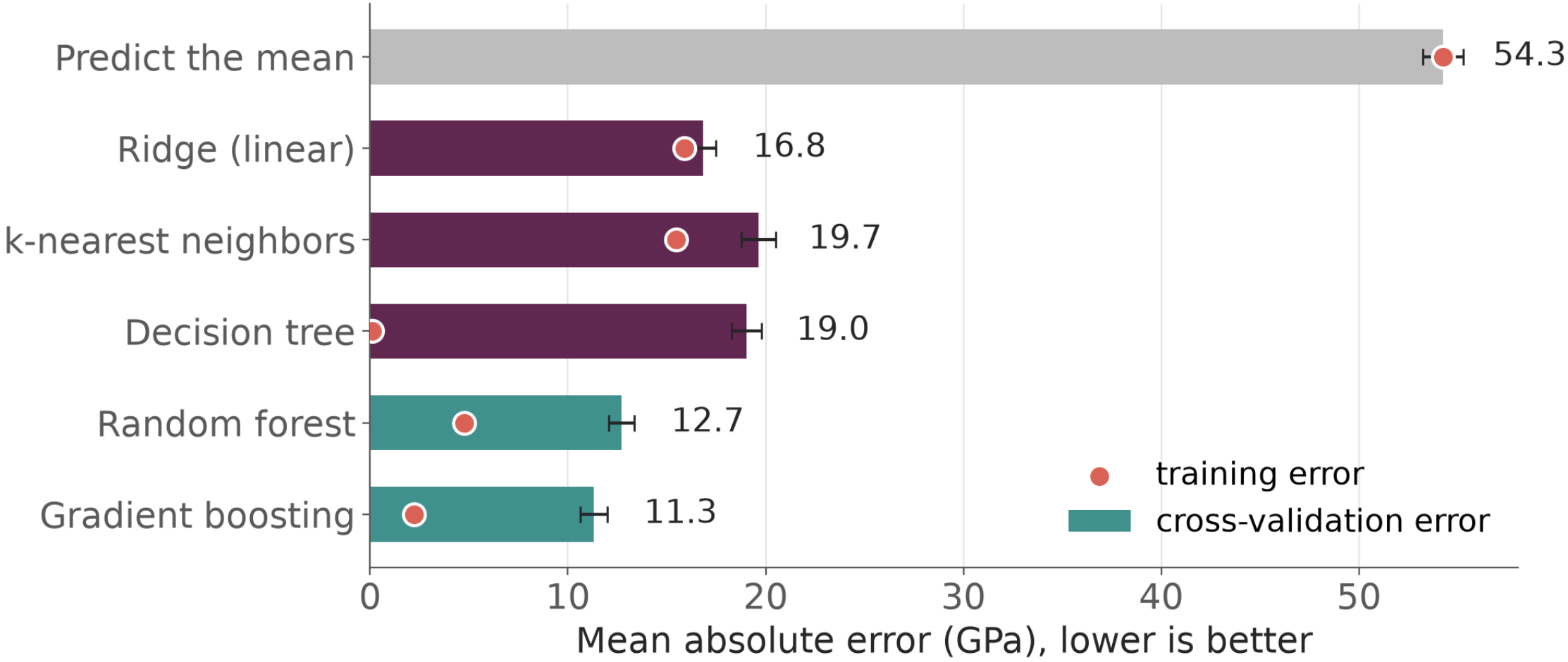
Gradient boosting
Often most accurate, more knobs

Gaussian process
Small data + uncertainty (Session 5)

Try several and let cross-validation decide

Tree ensembles win while nearly memorizing the training set

5-fold cross-validation on 3,898 training compounds, 264 Oliynyk CBFV features



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

How good is it?

Metrics for regression and classification

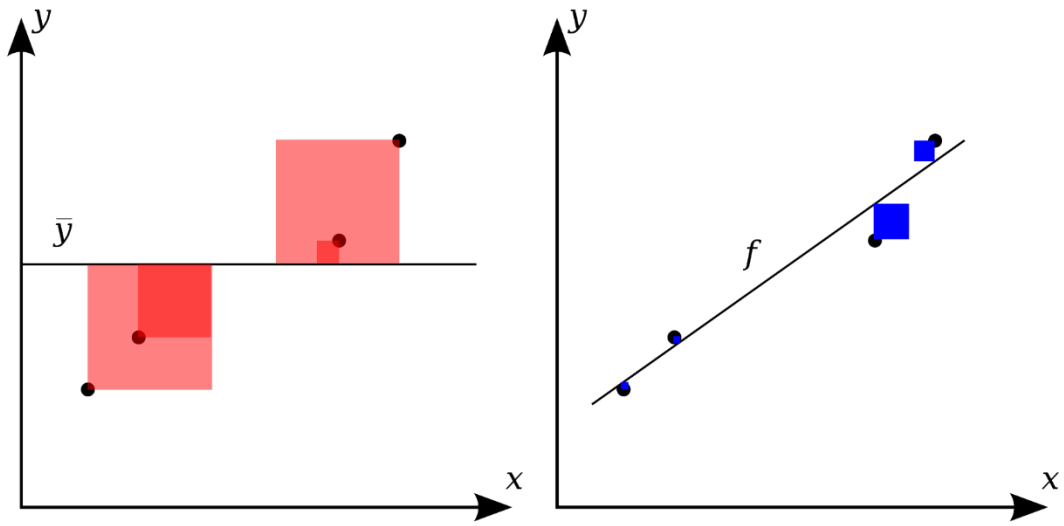
Most popular is Coefficient of Determination, R^2

$$\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

$$SS_{res} = \sum_i (y_i - f_i)^2$$

$$SS_{total} = \sum_i (y_i - \bar{y})^2$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{total}}$$



R^2 is the <u>proportion</u> of the variation in the dependent variable that is attributable to the independent variables

https://en.wikipedia.org/wiki/Coefficient_of_determination

Report errors in the units of the property, with a parity plot

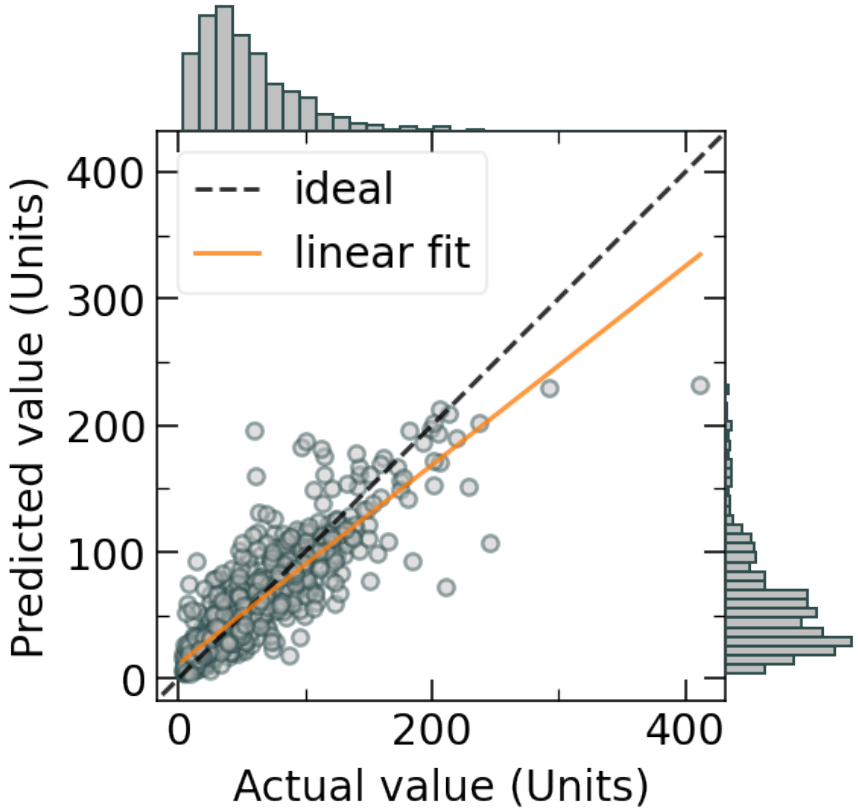
Parity plots

- Include histograms on axes to show point density
- Semi-transparent points also helps
- Add 1:1 line for perfect fit, plus a best fit

$$Mean\ absolute\ error, MAE = \frac{\sum_{i=1}^{n}|y_i - f_i|}{n}$$

$$Mean\ squared\ error, MSE = \frac{\sum_{i=1}^{n}(y_i - f_i)^2}{n}$$

$$Root\ mean\ squared\ error, RMSE = \sqrt{\frac{\sum_{i=1}^{n}(y_i - f_i)^2}{n}}$$

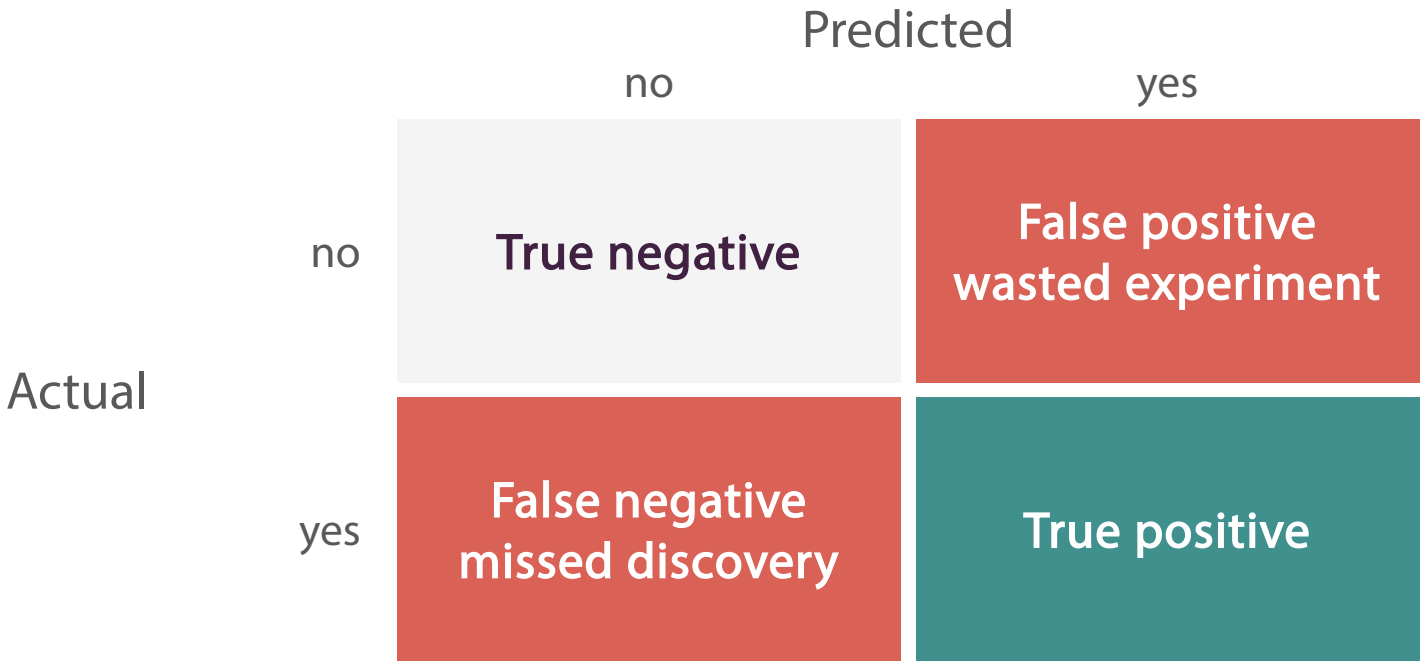


R^2 depends on the spread of the data; MAE doesn't

All compounds		Only 50 to 150 GPa	
MAE 11.6 GPa	R^2 0.92	MAE 9.6 GPa	R^2 0.68

Same model, same kind of errors: quote MAE with units, and compare R^2 only on the same data

A confusion matrix shows every kind of right and wrong



Recall
found / all real positives

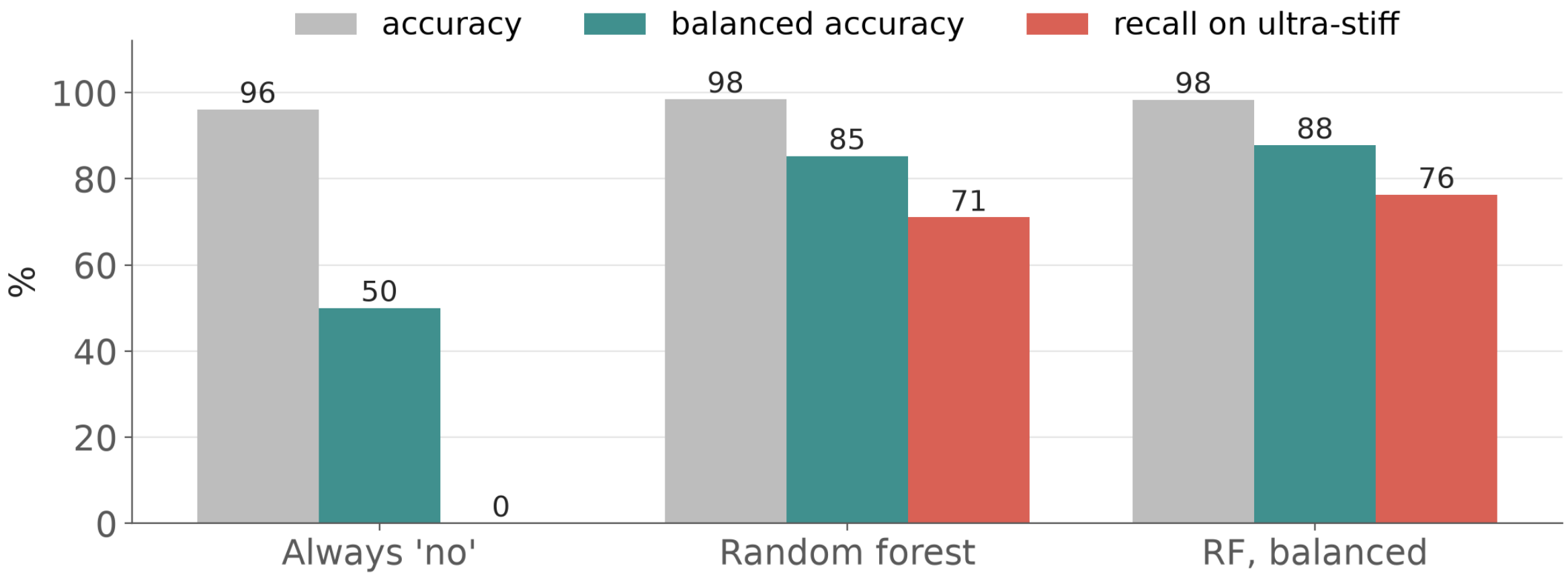
Precision
real / all flagged

Balanced accuracy
mean recall of each class

Accuracy alone hides which mistakes you're making

Always guessing 'no' is 96% accurate and finds no ultra-stiff compounds

Ultra-stiff: K ≥ 250 GPa, 3.9% of compounds. 38 of 975 test compounds are positive

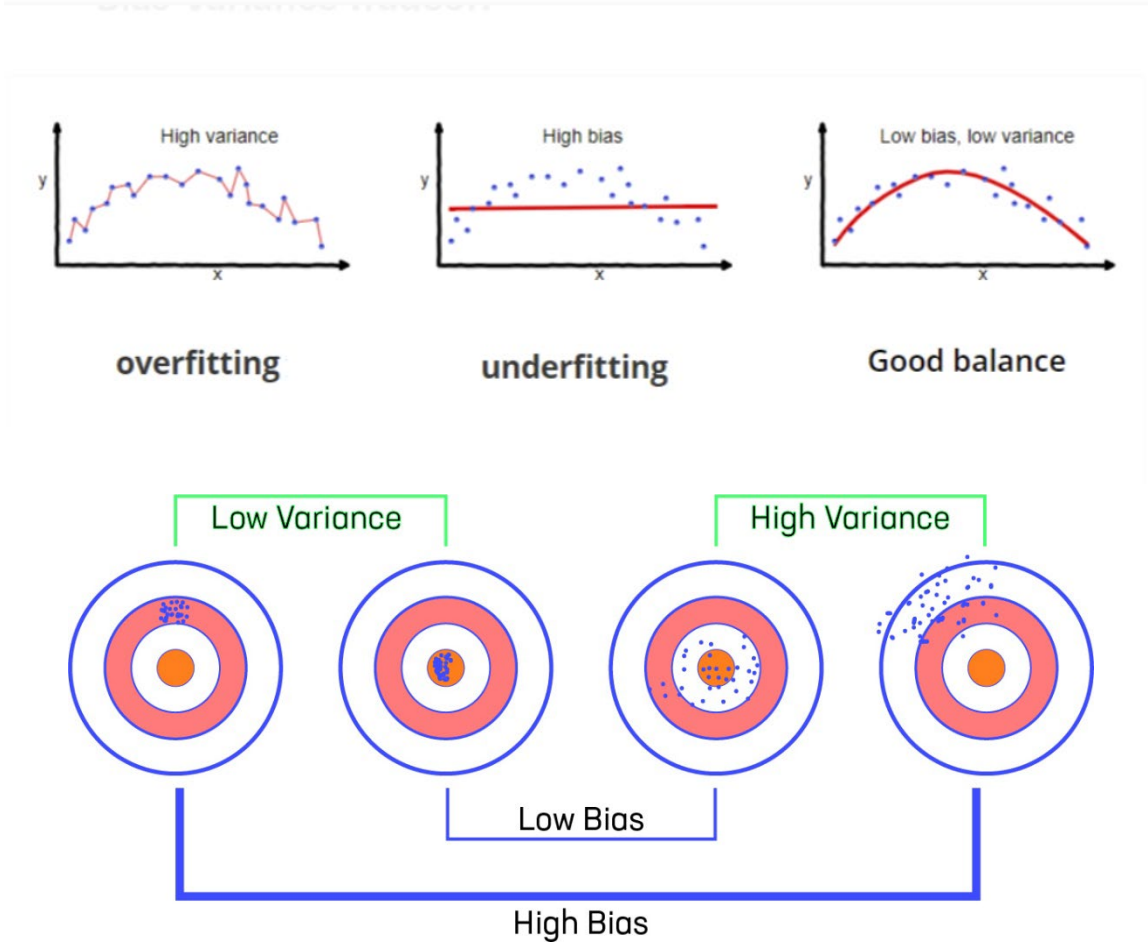


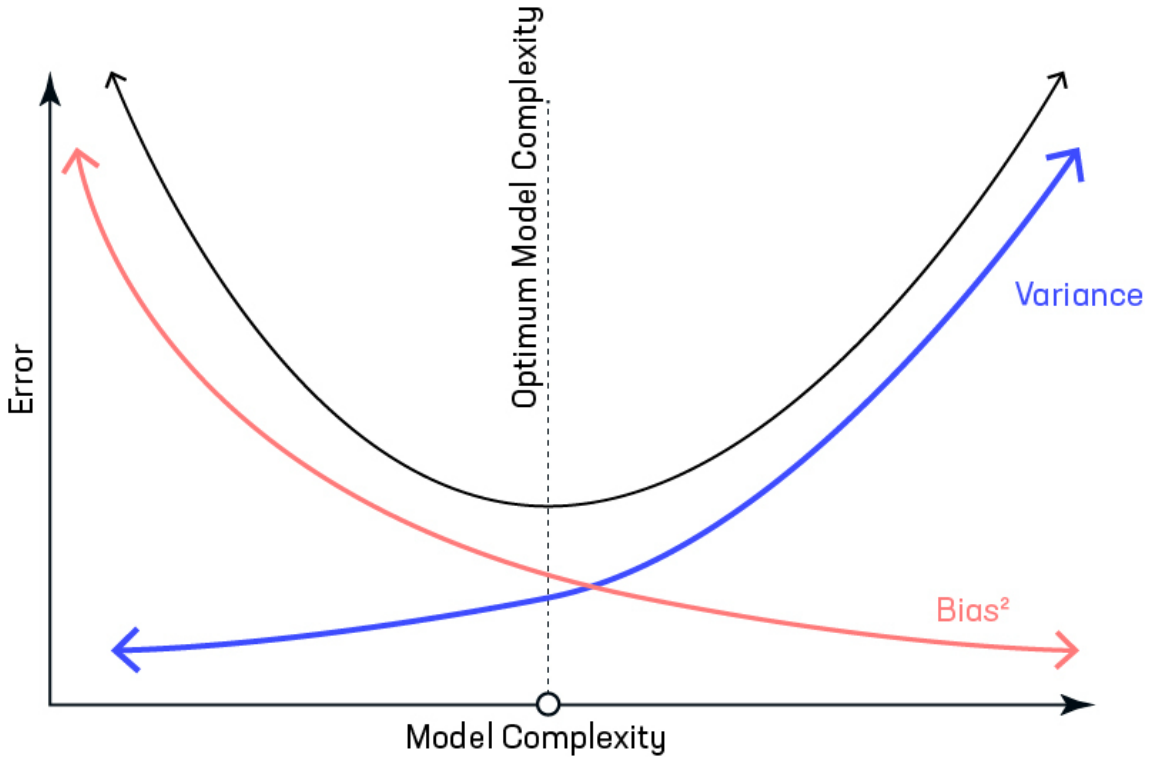
Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Overfitting and hyperparameters

Let validation data choose the knobs, never the test set

Too simple underfits, too flexible memorizes the noise

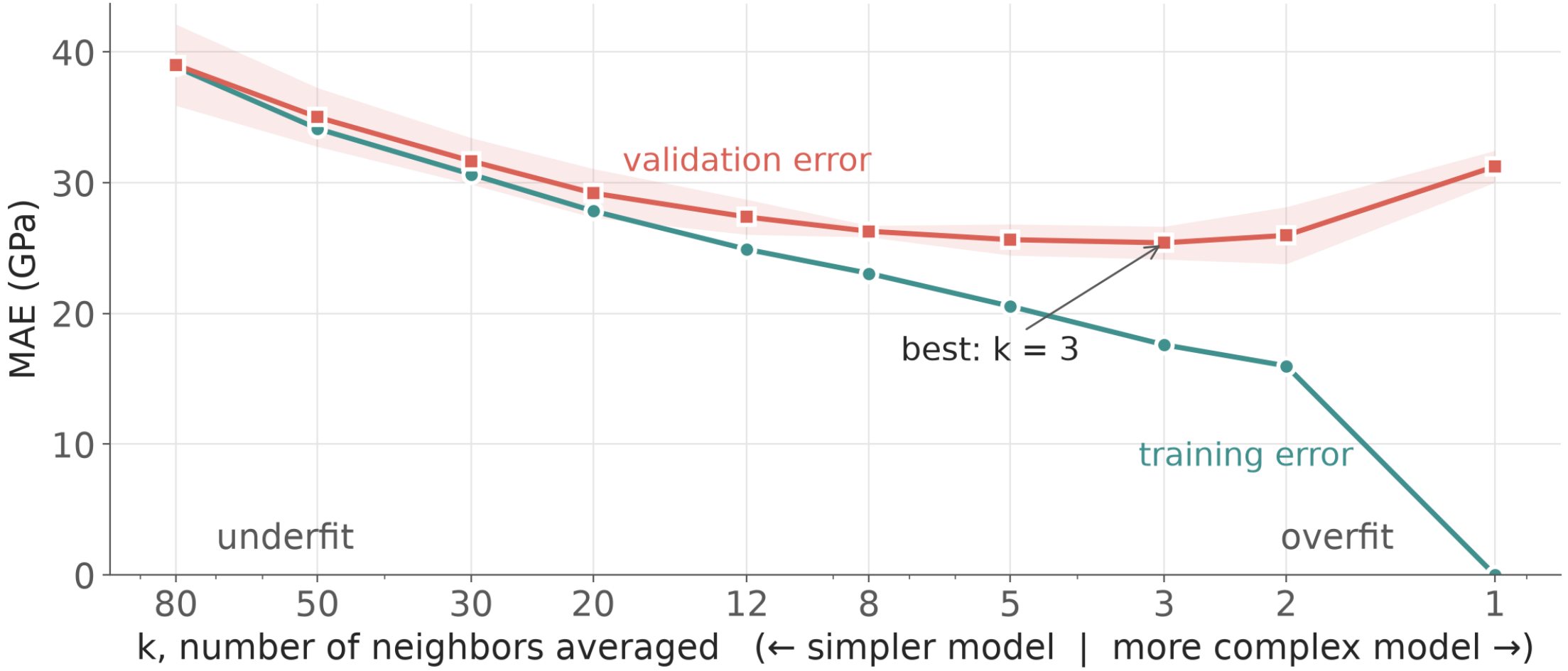




https://www.geeksforgeeks.org/lasso-vs-ridge-vs-elastic-net-ml/

Validation error shows where underfitting turns into overfitting

k-nearest neighbors on 300 compounds: k = 1 has zero training error and is far from the best model



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Regularization penalizes complexity to fight overfitting

Ridge regression introduces a penalty term based on the square of the coefficients plus a coefficient to control the penalty term. AKA L2 regularization

$$L_{ridge} = argmin_{\hat{\beta}}(\|Y - \beta * X\|^2 + \lambda\|\beta\|_2^2)$$

Lasso regression introduces a penalty term based on the sum of the coefficients plus a coefficient to control the penalty term. AKA L1 regularization

$$L_{lasso} = argmin_{\hat{\beta}}(\|Y - \beta * X\|^2 + \lambda\|\beta\|_1)$$

Elastic Net combines both.

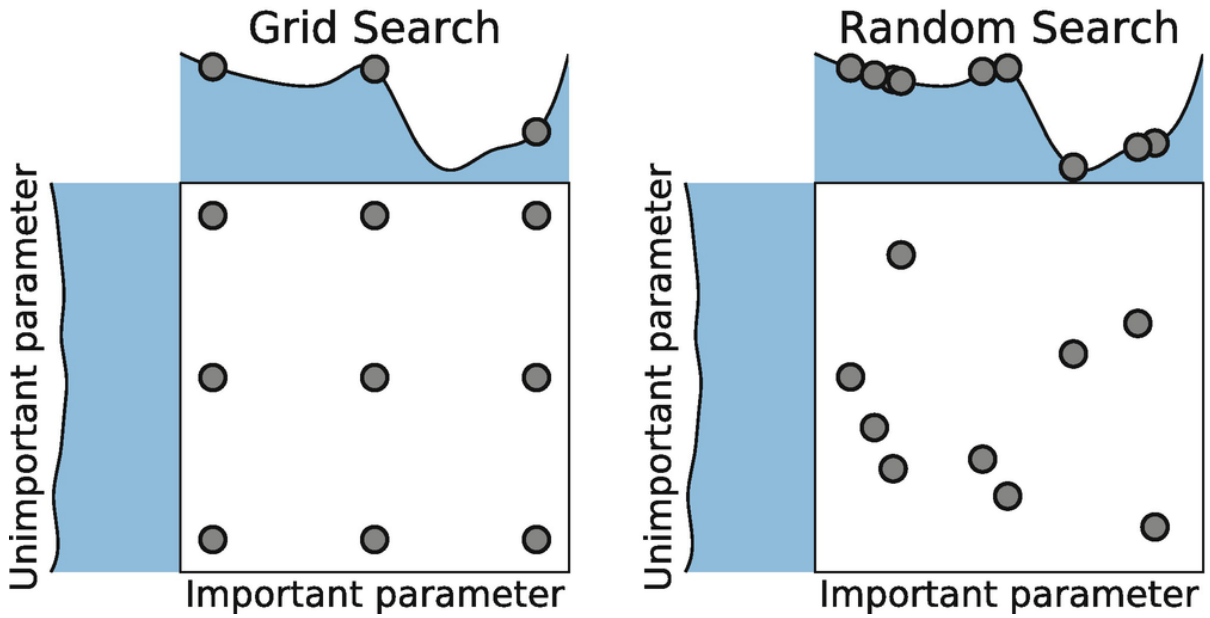
https://www.geeksforgeeks.org/lasso-vs-ridge-vs-elastic-net-ml/

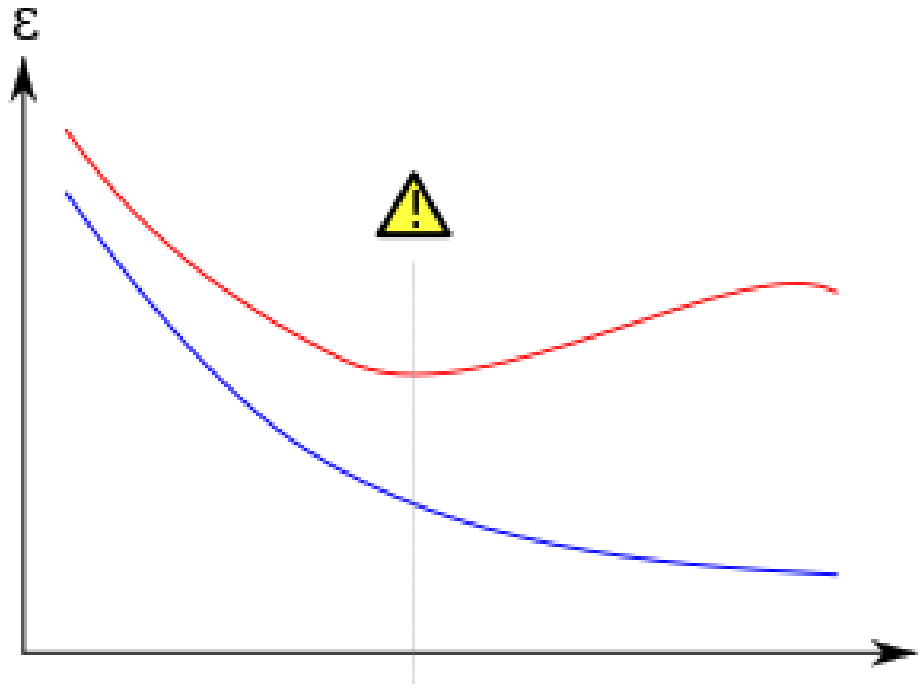
Parameters are learned; hyperparameters are chosen by you

Model	Knobs worth tuning first
Ridge / Lasso	alpha (penalty strength)
k-nearest neighbors	n_neighbors, weights
Random forest	max_features, min_samples_leaf, max_depth (n_estimators: more is just slower)
Gradient boosting	learning_rate with max_iter, max_leaf_nodes, min_samples_leaf
Gaussian process	the kernel and its length scales (Session 5)

Every knob you turn while watching a score is a chance to fit the noise

Grid and random search let validation scores choose the knobs

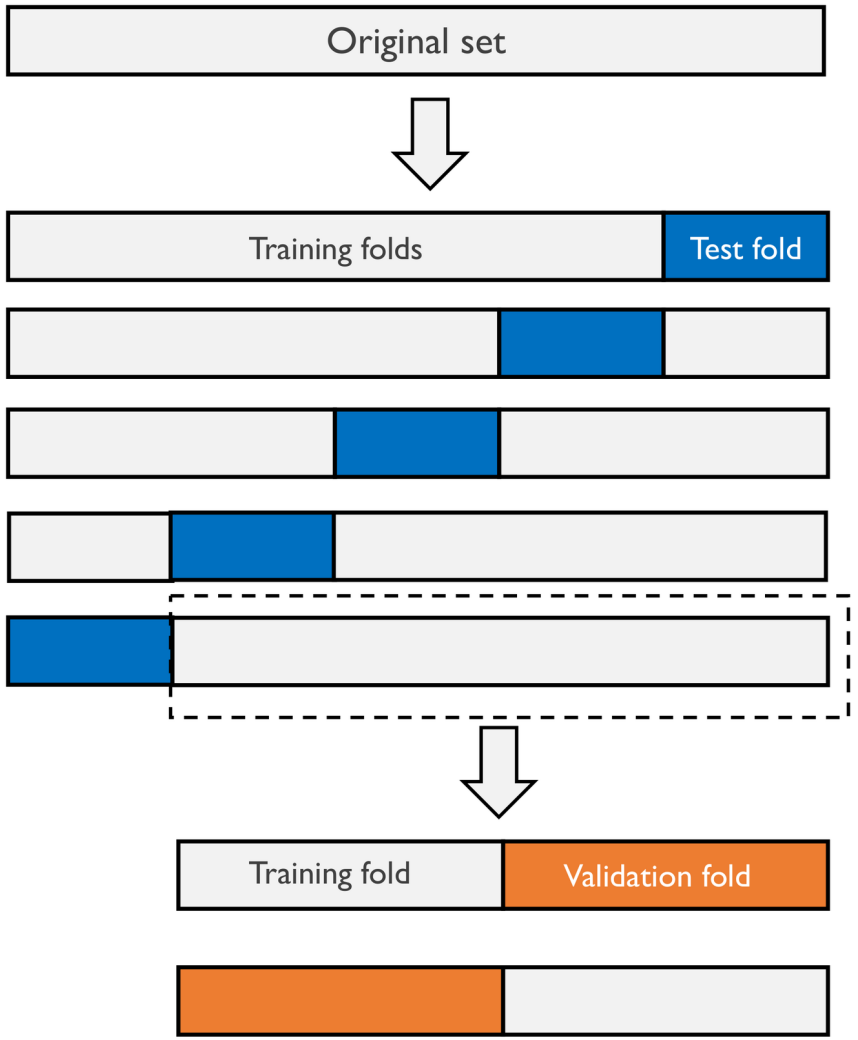




https://en.wikipedia.org/wiki/Overfitting

Tune in the inner loop and score only in the outer loop

Pictured: 5-fold outer CV for evaluation, 2-fold inner CV for hyperparameter tuning.



Outer loop

Train with optimal parameters from the inner loop; then average as an estimate of the generalization performance

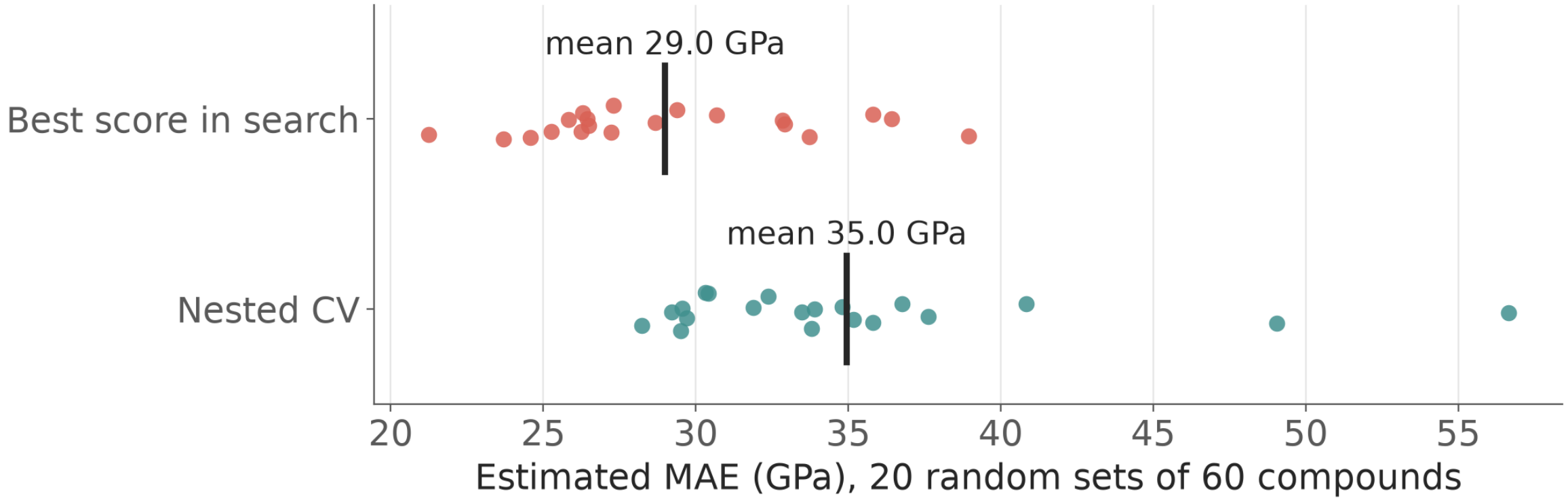
Inner loop

Tune parameters

Raschka, arXiv:1811.12808 (2018) https://arxiv.org/abs/1811.12808

The best score from a search is optimistic; nested CV is honest

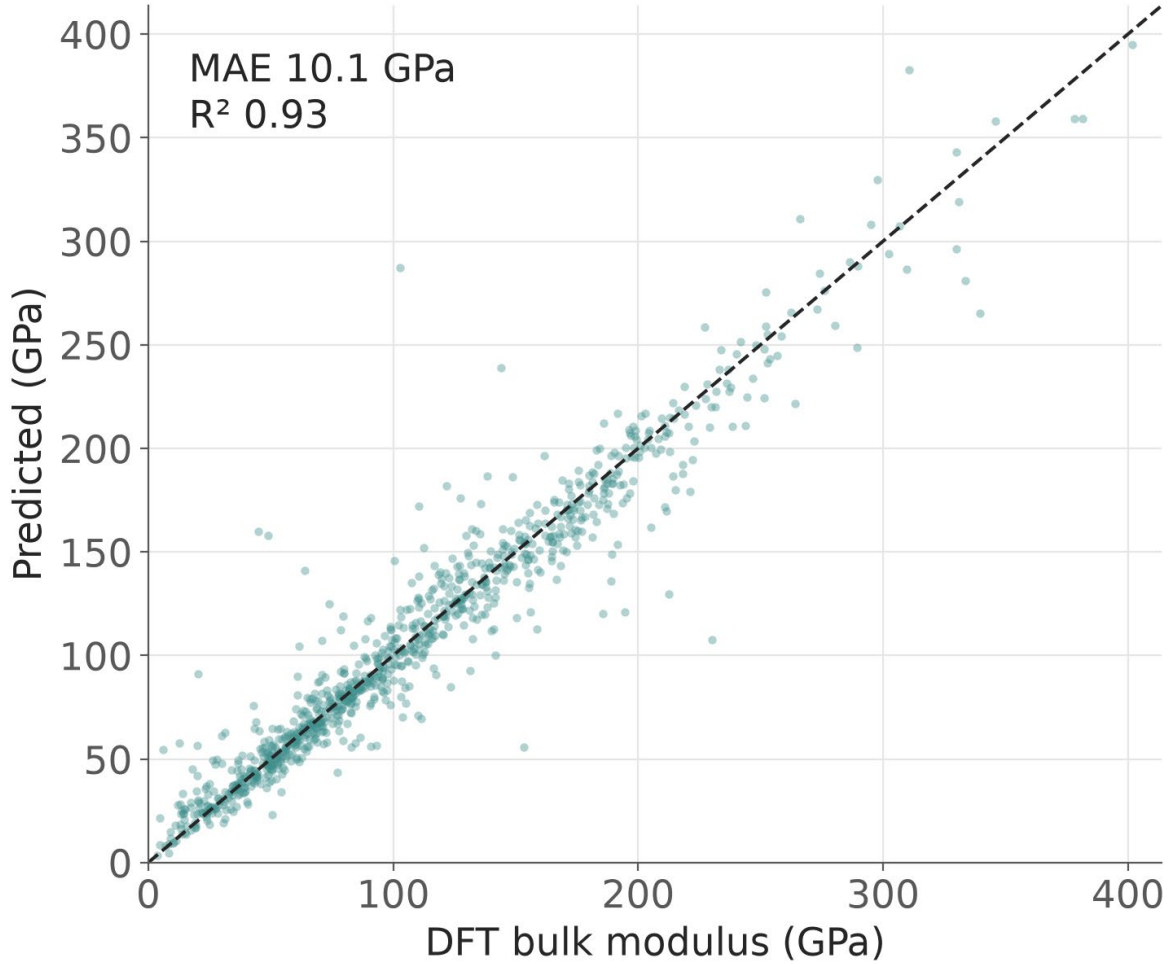
98 settings (number of features × k neighbors) searched on 20 random sets of 60 compounds



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Only at the end: one honest look at the test set

Gradient boosting, chosen with CV on training data, then scored once: test MAE 10.1 GPa



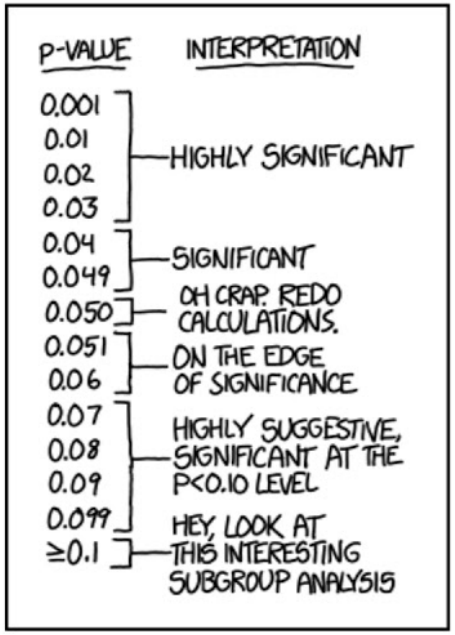
Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

How to fool yourself

p-hacking, lucky splits and leakage

We need a test set to prevent p-hacking!

What is p-hacking?



https://imgs.xkcd.com/comics/p_values.png

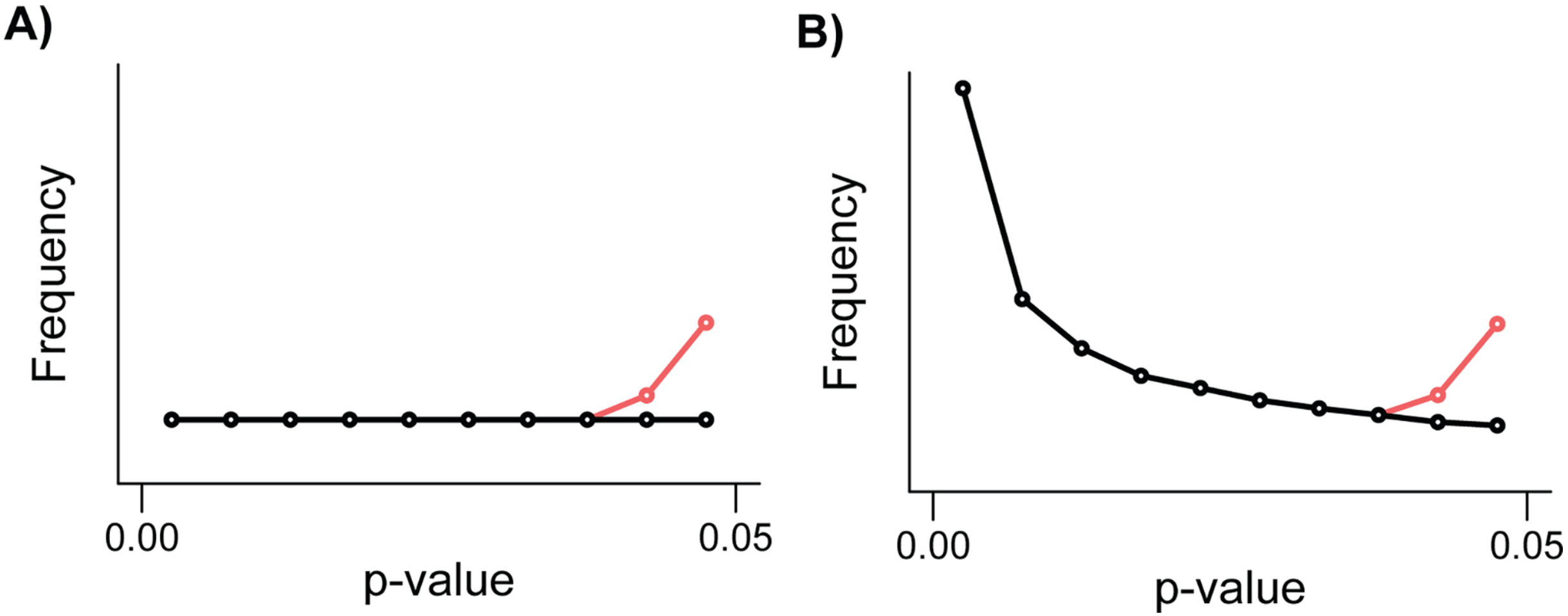


There are many ways to p-hack

1. Stop collecting data once $p<.05$
2. Analyze many measures, but report only those with $p<.05$.
3. Collect and analyze many conditions, but only report those with $p<.05$.
4. Use covariates to get $p<.05$.
5. Exclude participants to get $p<.05$.
6. Transform the data to get $p<.05$.

P-hacking leaves a telltale bump of p-values just below 0.05

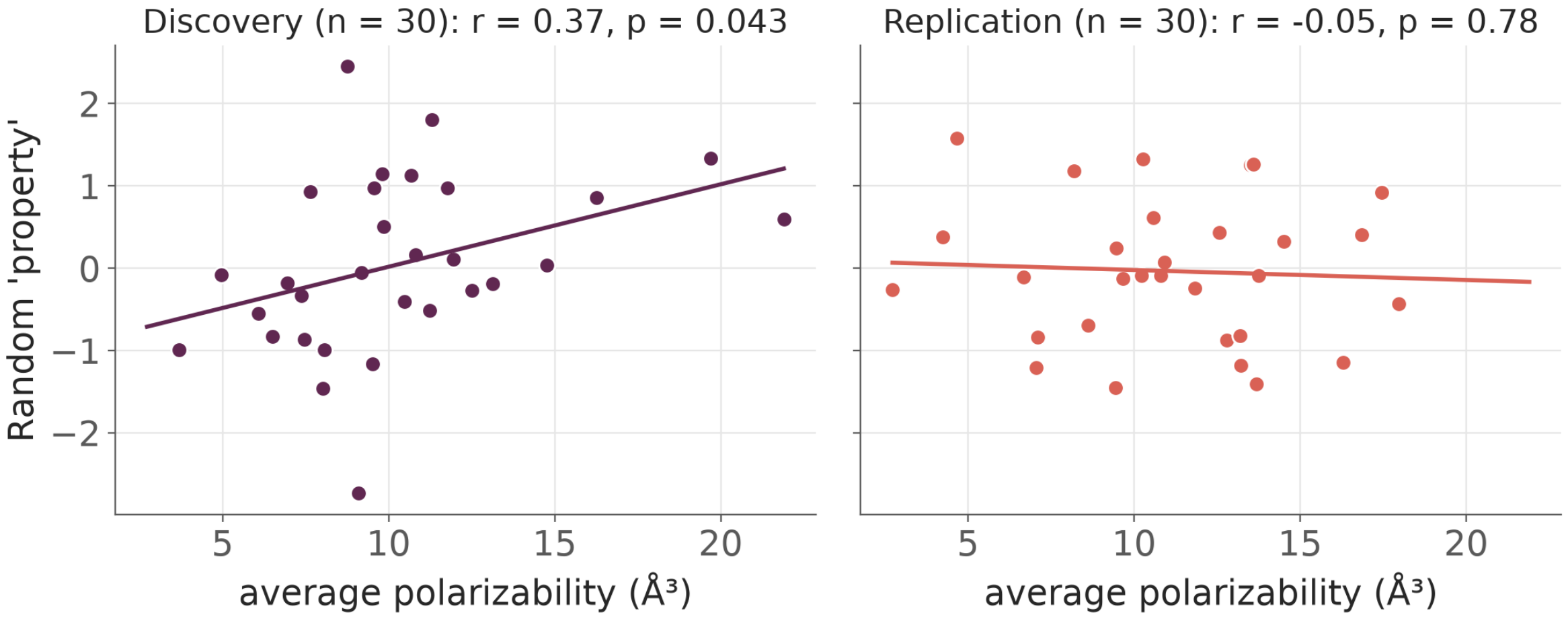
Across 14 disciplines, 54.6% of p-values in the 0.04-0.05 range fell in the 0.045-0.05 bin.



Head et al., PLoS Biol. 13, e1002106 (2015) https://doi.org/10.1371/journal.pbio.1002106

On pure noise, 80% of small studies 'find' a significant feature

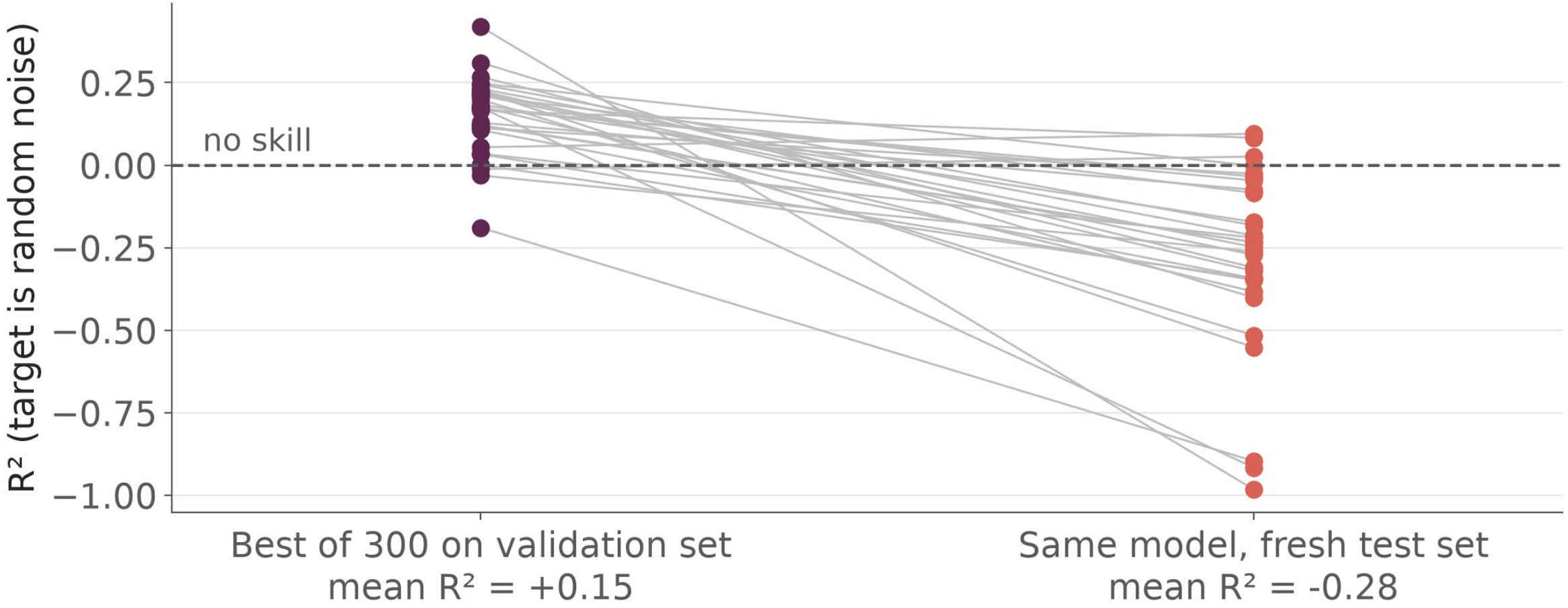
Example: p = 0.043 in 30 compounds, r = -0.05 in 30 new ones (about 173 features tested)



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Try 300 models on noise, keep the best, and you'll 'find' a model

Each line: the winner of 300 models (features, type, settings) on a 20-compound validation set



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Every choice made by looking at a score becomes part of training

Random seed
Re-split until it looks good

Features
Keep the subset that scores best

Outliers
Drop the points that hurt

Models
Report only the winner of many

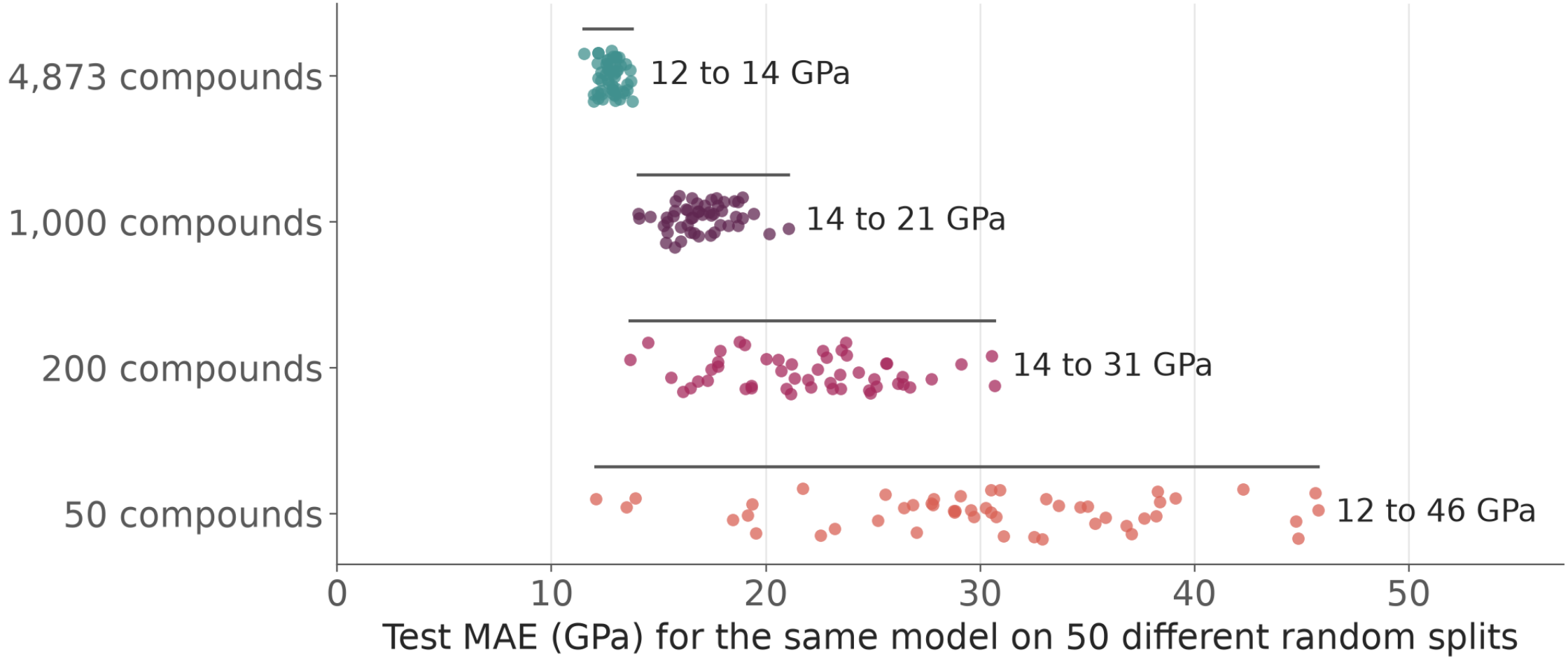
Hyperparameters
Tune on the test set

Metric
Pick R^2, MAE or RMSE, whichever looks best

Decide how you'll validate before you look at any score

With 50 compounds, one model's test error ranges from 12 to 46 GPa

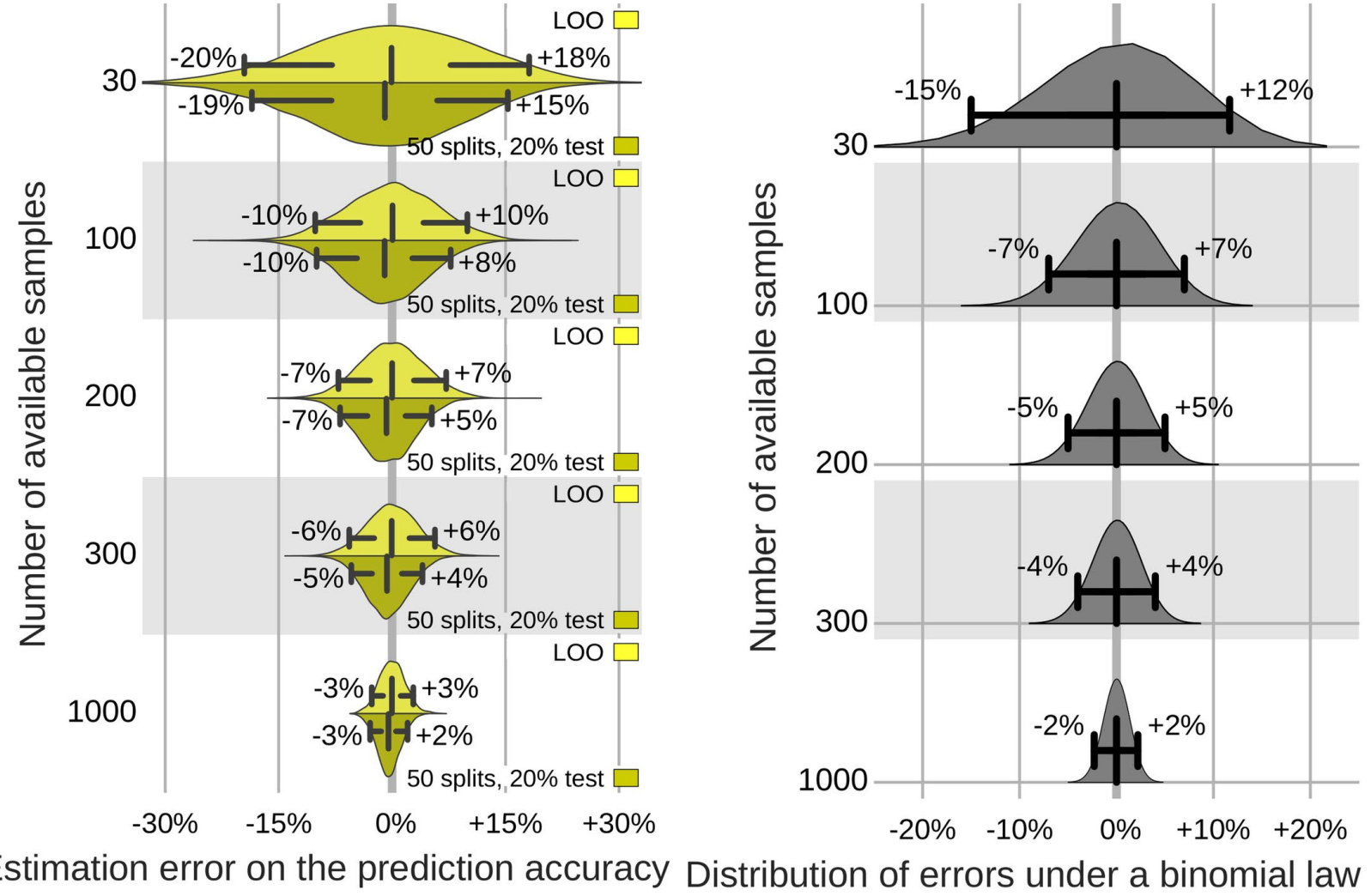
Same model, same data source: only the random train/test split changes



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

With 100 samples, a CV accuracy estimate carries about ±10% error

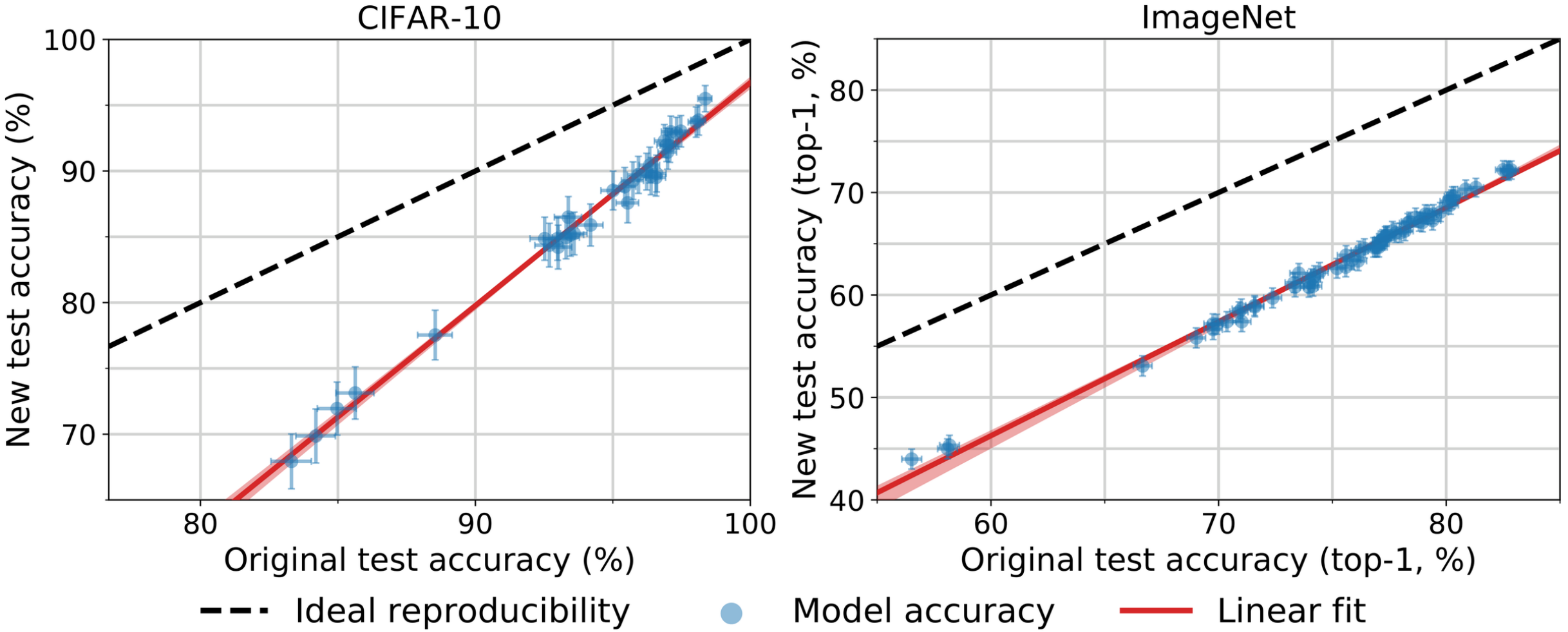
The paper's Table 1 gives ±15%, 10%, 6% and 3% bounds for n = 30, 100, 300 and 1000.



Varoquaux, NeuroImage 180, 68-77 (2018); arXiv:1706.07581 https://arxiv.org/abs/1706.07581

A freshly collected test set cost top image models 11-14% accuracy

Rankings held, but every model dropped: your test set is one sample of the world, not the world



Recht, Roelofs, Schmidt & Shankar, ICML 2019 (PMLR 97); arXiv:1902.10811 https://arxiv.org/abs/1902.10811

Leakage

When the test set sneaks into training

Data leakage has been documented in 294 papers across 17 fields

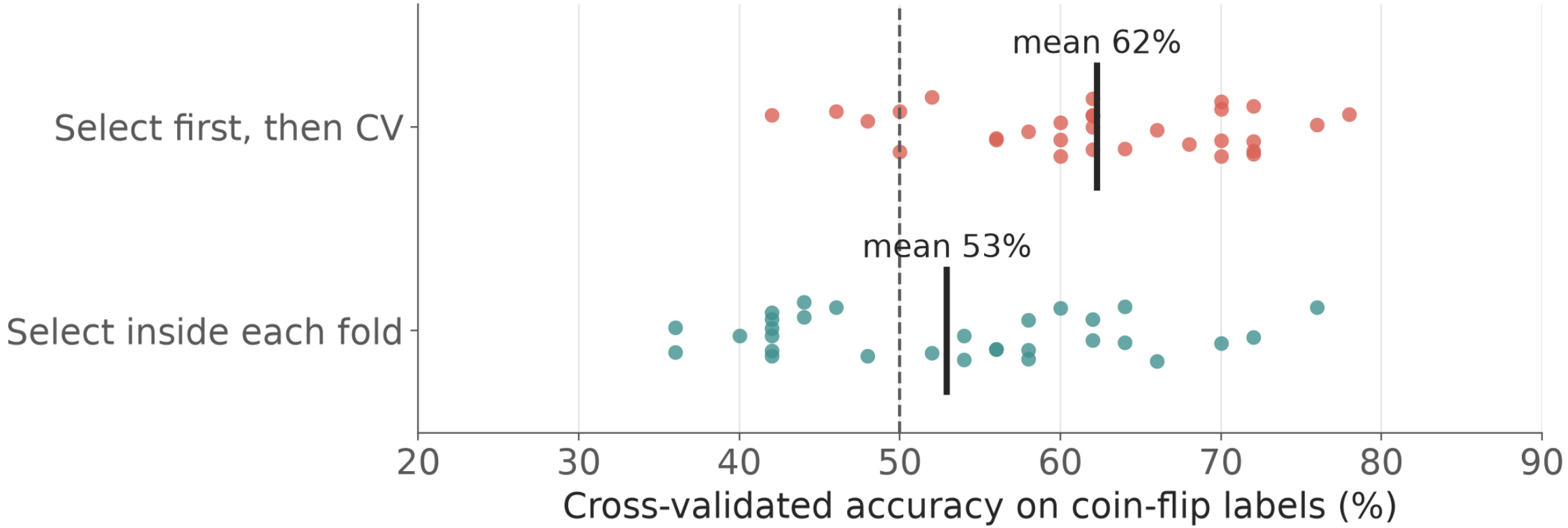
Kapoor & Narayanan sort leakage into eight types, from 'no test set' to sampling bias.

Field	Paper	Number of papers reviewed	Number of papers with pitfalls	**[L1.1] No test set**	**[L1.2] Pre-proc. on train-test**	**[L1.3] Feature sel. on train-test**	**[L1.4] Duplicates**	**[L2] Illegitimate features**	**[L3.1] Temporal leakage**	**[L3.2] Non-ind. b/w train-test**	**[L3.3] Sampling bias**	Comput. reproducibility issues	Data quality issues	Metric choice issues	Standard dataset used?
Medicine	Bouwmeester et al. (2012)	71	27	○									○		
Neuroimaging	Whelan & Garavan (2014)	–	14	○		○									
Bioinformatics	Blagus & Lusa (2015)	–	6		○										
Autism Diagnostics	Bone et al. (2015)	–	3				○			○			○	○	○
Nutrition Research	Ivanescu et al. (2016)	–	4	○									○	○	
Software Eng.	Tu et al. (2018)	58	11					○			○		○		○
Toxicology	Alves et al. (2019)	–	1				○				○		○		
Clinical Epidem.	Christodoulou et al. (2019)	71	48			○							○		
Satellite Imaging	Nalepa et al. (2019)	17	17						○				○		○
Tractography	Poulin et al. (2019)	4	2	○							○		○	○	○
Brain-computer Int.	Nakanishi et al. (2020)	–	1	○											○
Histopathology	Oner et al. (2020)	–	1						○						
Neuropsychiatry	Poldrack et al. (2020)	100	53	○	○								○	○	
Neuroimaging	Ahmed et al. (2021)	–	1						○						
Neuroimaging	Li et al. (2021)	122	18						○						
IT Operations	Lyu et al. (2021)	9	3					○							○
Medicine	Filho et al. (2021)	–	1				○								
Radiology	Roberts et al. (2021)	62	16	○			○			○	○				○
Neuropsychiatry	Shim et al. (2021)	–	1			○					○				
Medicine	Vandewiele et al. (2021)	24	21			○			○	○	○		○		○
Computer Security	Arp et al. (2022)	30	22	○	○	○		○	○	○			○	○	
Genomics	Barnett et al. (2022)	41	23			○							○		

Kapoor & Narayanan, Patterns 4, 100804 (2023) https://doi.org/10.1016/j.patter.2023.100804

Selecting features before the split finds signal in coin flips

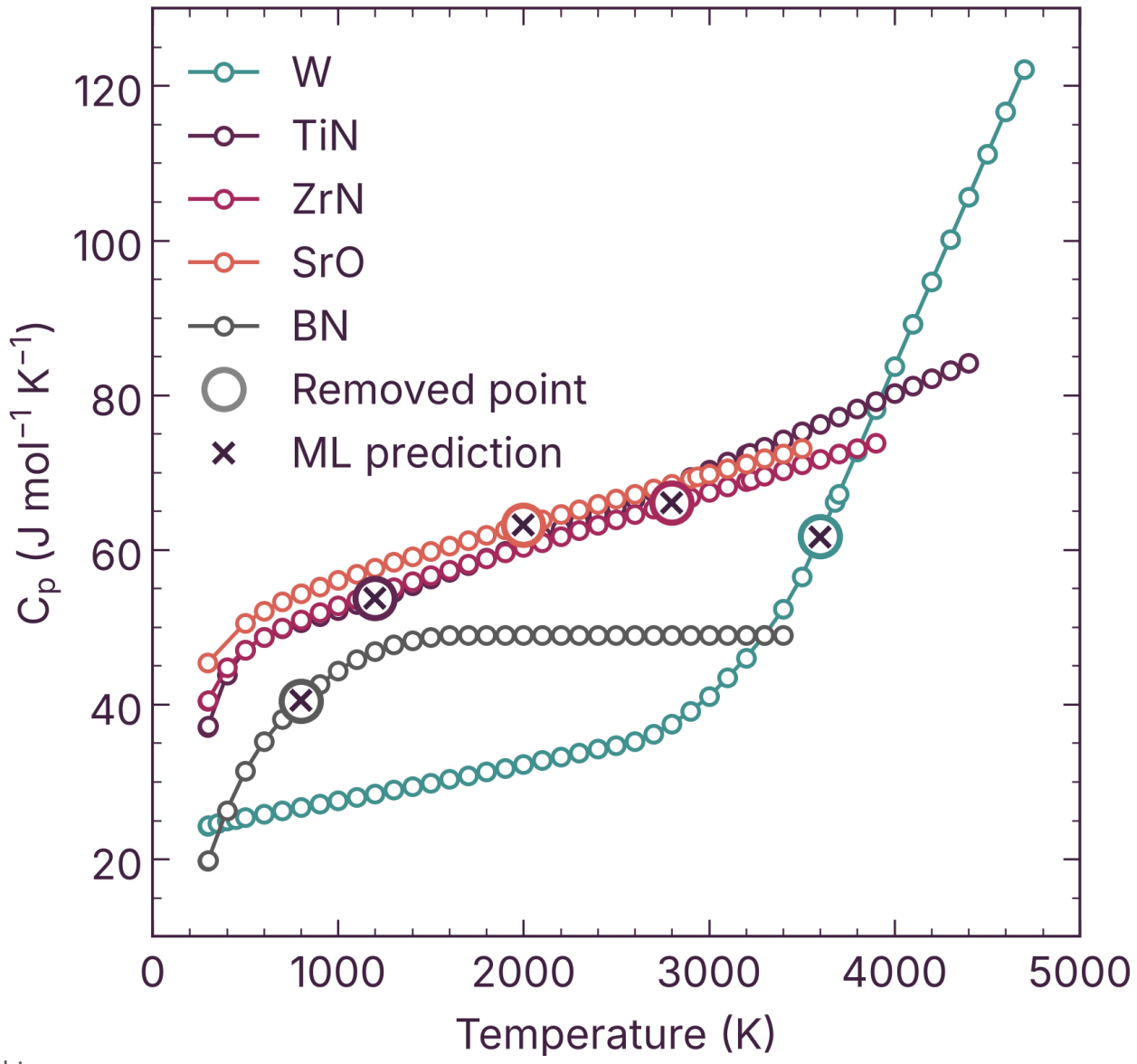
50 compounds, random 0/1 labels, keep the 10 best of 264 features. The truth is 50%



Course data: AFLOW bulk moduli (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Careful that your code matches your use case

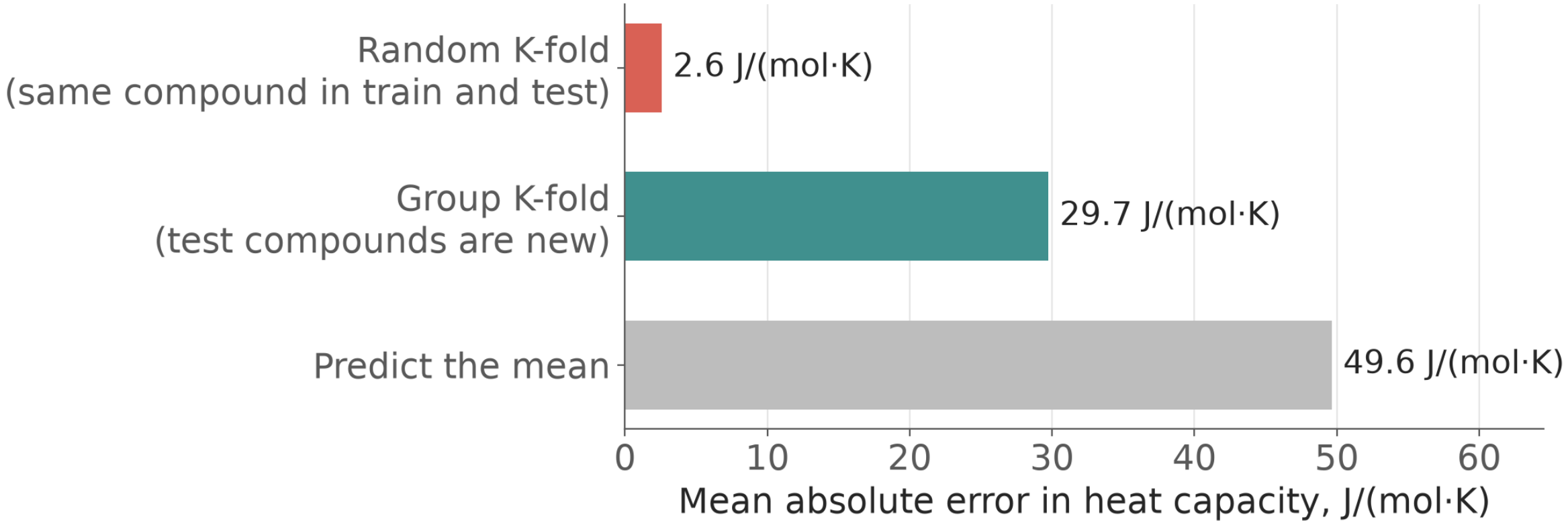
If you want to build a model to predict heat capacity at any temperature for a new compound, you need to hold out entire families of materials during training or you will only learn interpolation between temperatures of a family!



Course data: heat capacity vs temperature (Wang et al., Chem. Mater. 2020). Analysis made for this course.

The same compound on both sides of the split: 12× too optimistic

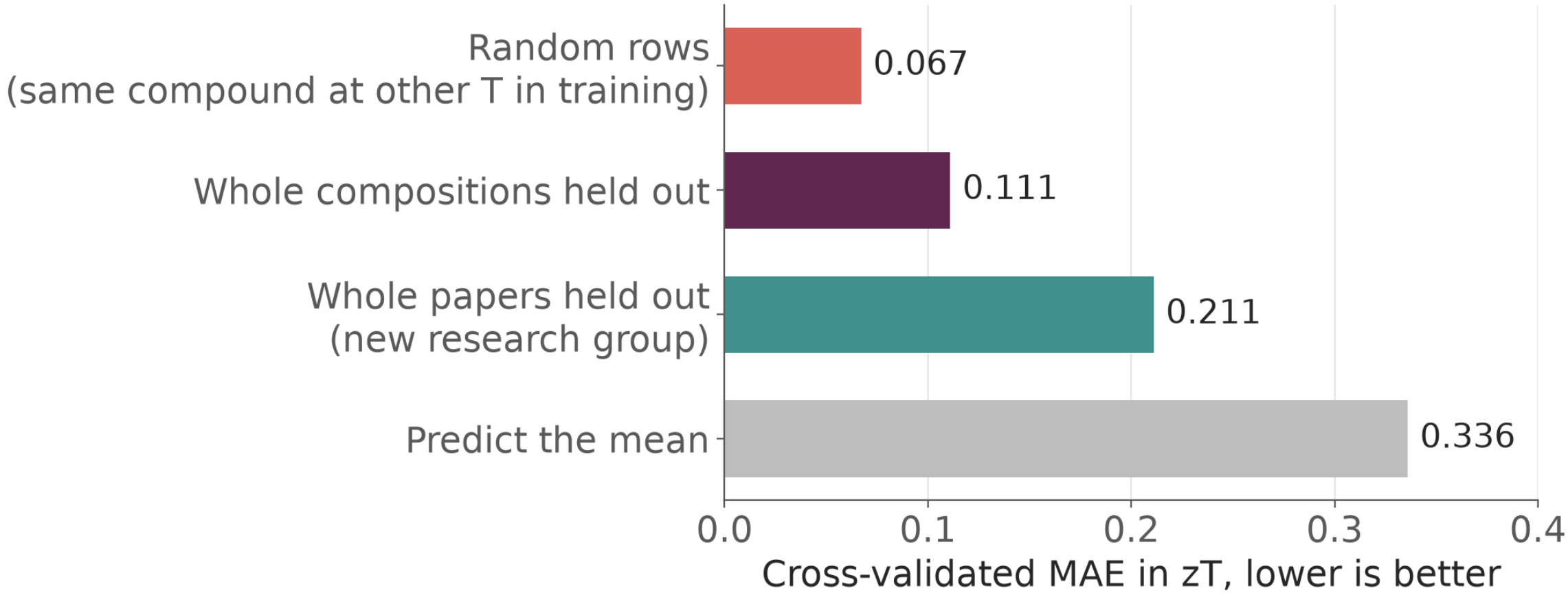
Heat capacity: 4,547 rows, but only 244 different compounds measured at many temperatures



Course data: heat capacity vs temperature (Wang et al., Chem. Mater. 2020). Analysis made for this course.

Thermoelectric models: the error depends on what 'new' means

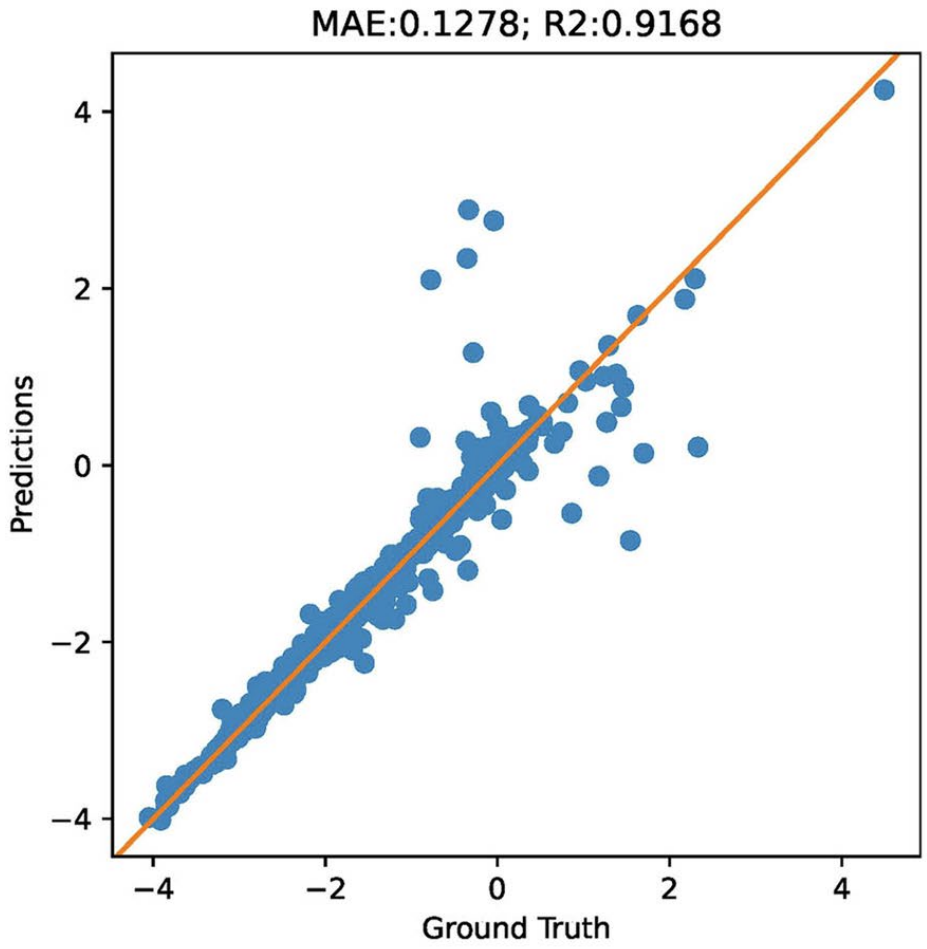
sysTEm experimental dataset: 6,333 zT measurements, composition + temperature, random forest



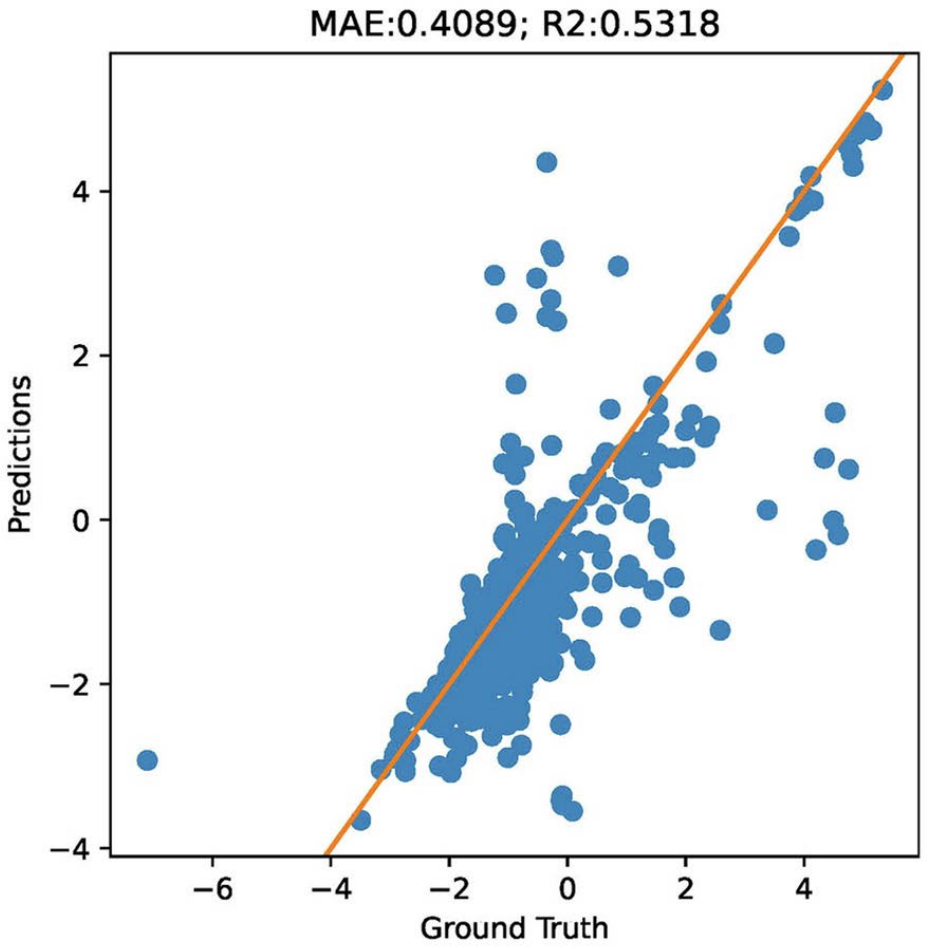
Data: Tang, Purdy, Mohanty, Ng & Sparks, Integr. Mater. Manuf. Innov. 15, 181 (2026) https://doi.org/10.1007/s40192-026-00446-5. Analysis made for this course.

Near-duplicates inflate random-split scores: R^2 0.92 fell to 0.53

Same model: R^2 0.917 on its random split, 0.532 on the most novel compounds



(c) Redundant Roost on random-split test set

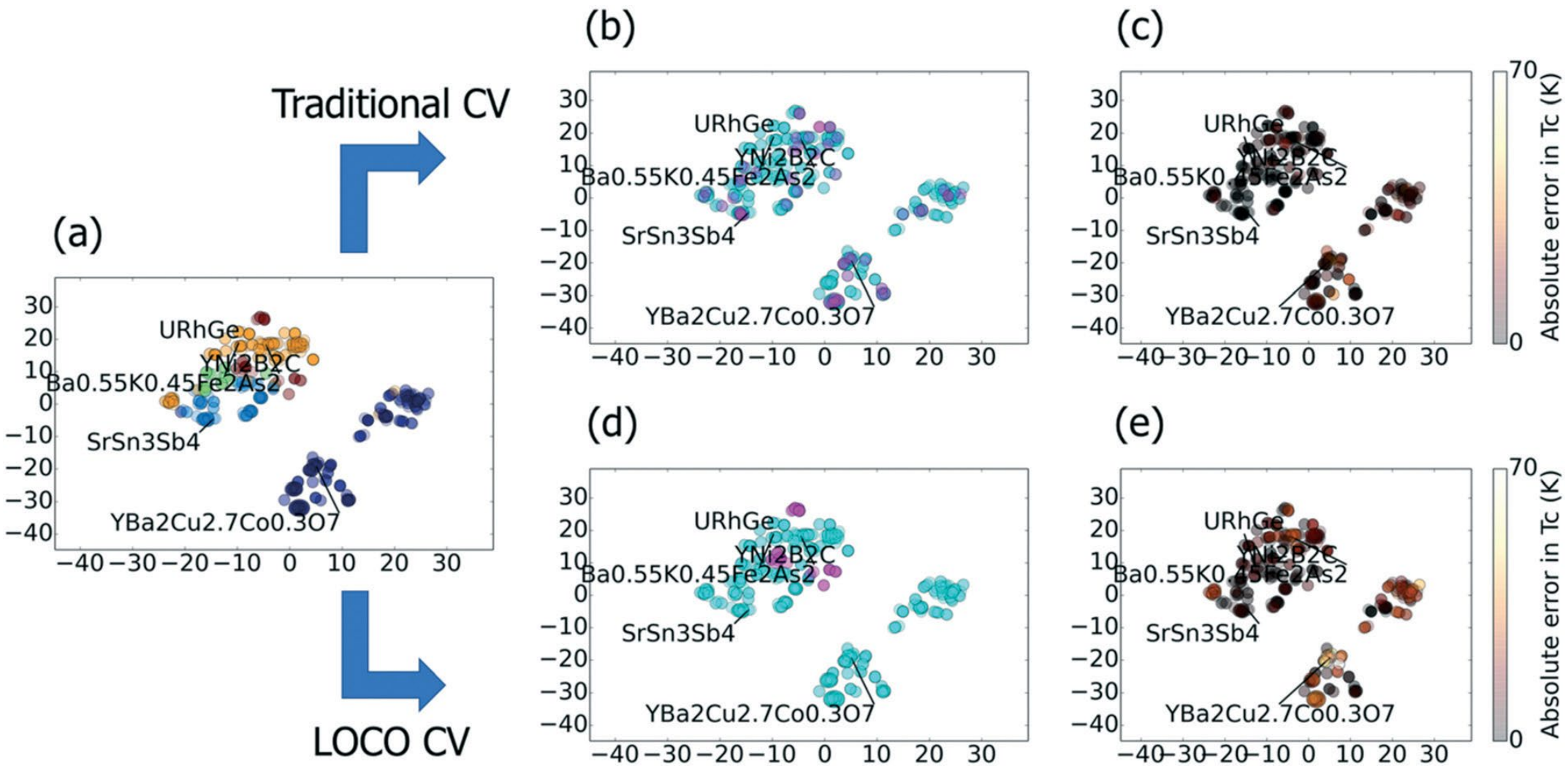


(d) Redundant Roost on OOD test set

Li, Q. et al., npj Comput. Mater. 10, 245 (2024) https://doi.org/10.1038/s41524-024-01426-z

Holding out whole chemical families exposes extrapolation error

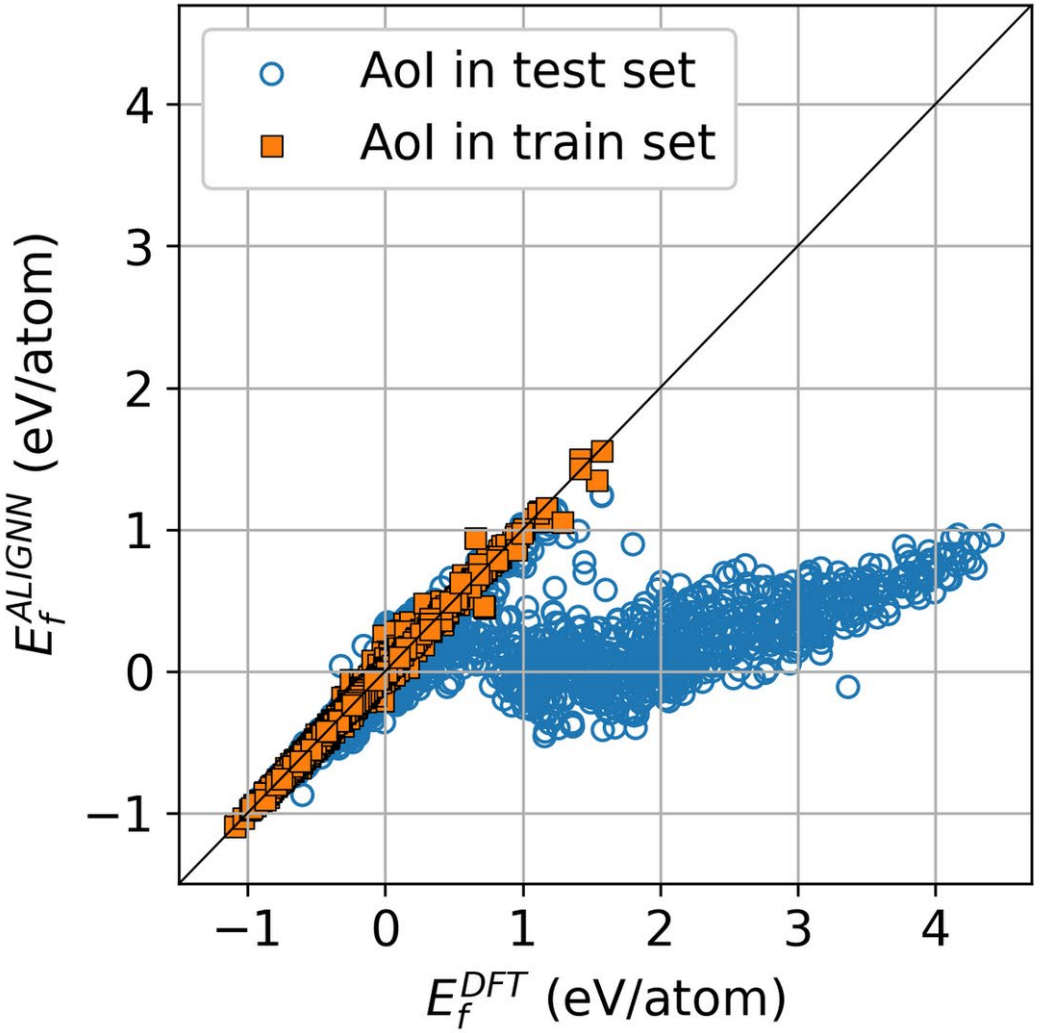
For log(Tc), random-forest median Pearson R falls from 0.87 (random CV) to 0.30 (LOCO CV).

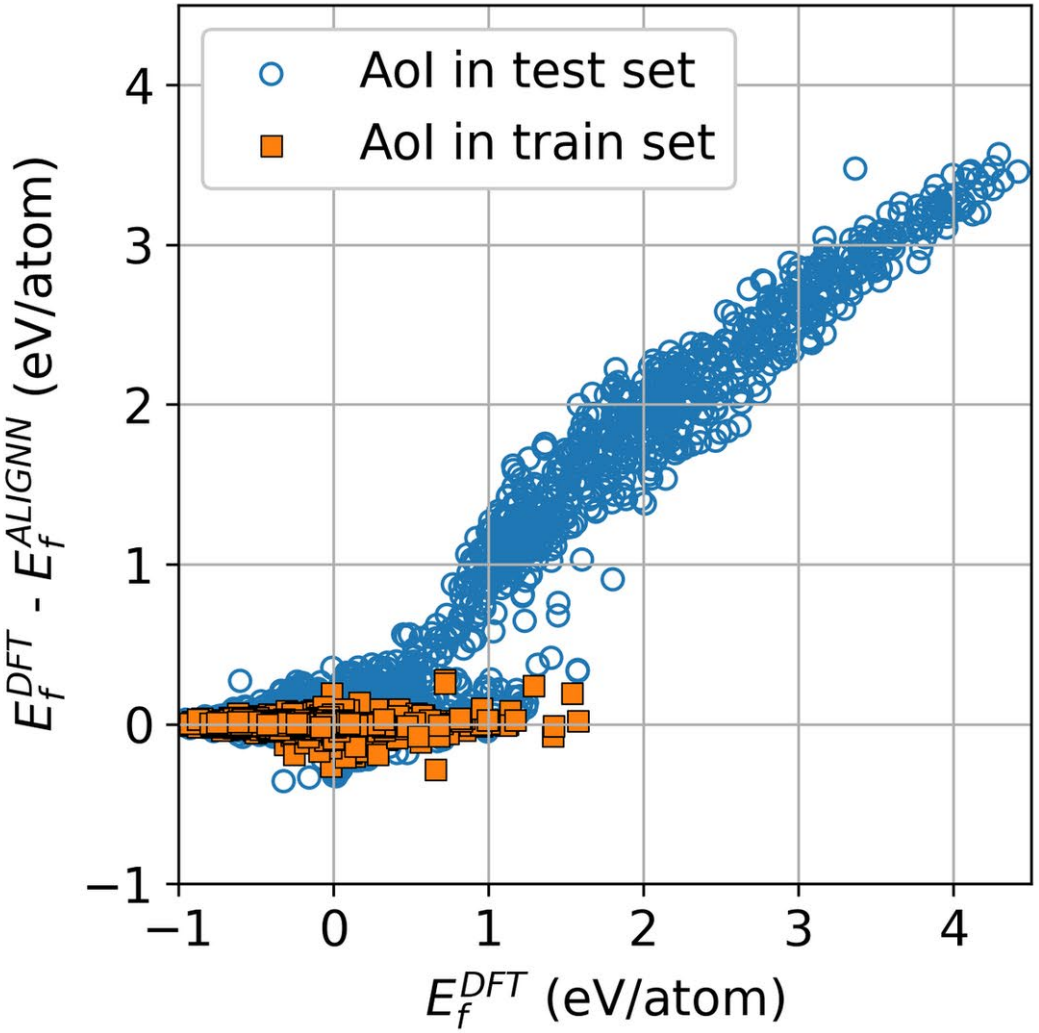


Meredig et al., Mol. Syst. Des. Eng. 3, 819-825 (2018) https://doi.org/10.1039/C8ME00012C

Many benchmarks are just overfits. Updated data shows new errors

MAE rose from 0.022 eV/atom on the MP18 test split to 0.297 eV/atom on the new MP21 alloys.





Li, K. et al., npj Comput. Mater. 9, 55 (2023) https://doi.org/10.1038/s41524-023-01012-9

Before you believe a number, check eight things

1. Test set split off first, used once
2. Beats a dummy by more than the fold spread
3. Mean ± spread, in units
4. Preprocessing inside a pipeline
5. Related rows grouped (GroupKFold)
6. Tuning on training data; nested CV if reporting it
7. Metric suits the class balance
8. Test set looks like the real use case

Validation > accuracy

Our best-practices guide walks through all of this for materials

ACS Publications
Most Trusted. Most Cited. Most Read.

Search text, DOI, authors, etc.

My Activity Publications

Machine Learning for Materials Scientists: An Introductory Guide toward Best Practices

Anthony Yu-Tung Wang, Ryan J. Murdock, Steven K. Kauwe, Anton O. Oliynyk, Aleksander Gurlo, Jakoah Brgoch, Kristin A. Persson, and Taylor D. Sparks*

Cite this: *Chem. Mater.* 2020, 32, 12, 4954–4965
Publication Date: May 19, 2020
https://doi.org/10.1021/acs.chemmater.0c01907
Copyright © 2020 American Chemical Society
RIGHTS & PERMISSIONS ✔Subscribed

Article Views 7626 | Altmetric 48 | Citations 45
LEARN ABOUT THESE METRICS

Share Add to Export RIS

Read Online | PDF (2 MB) | open URL | SI Supporting Info (1) »

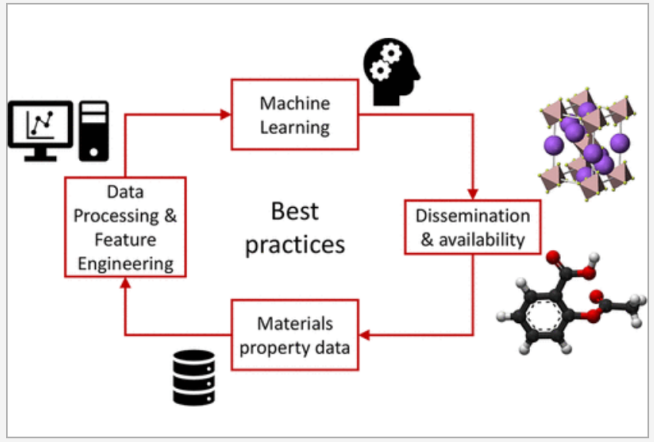
SUBJECTS: Materials science,



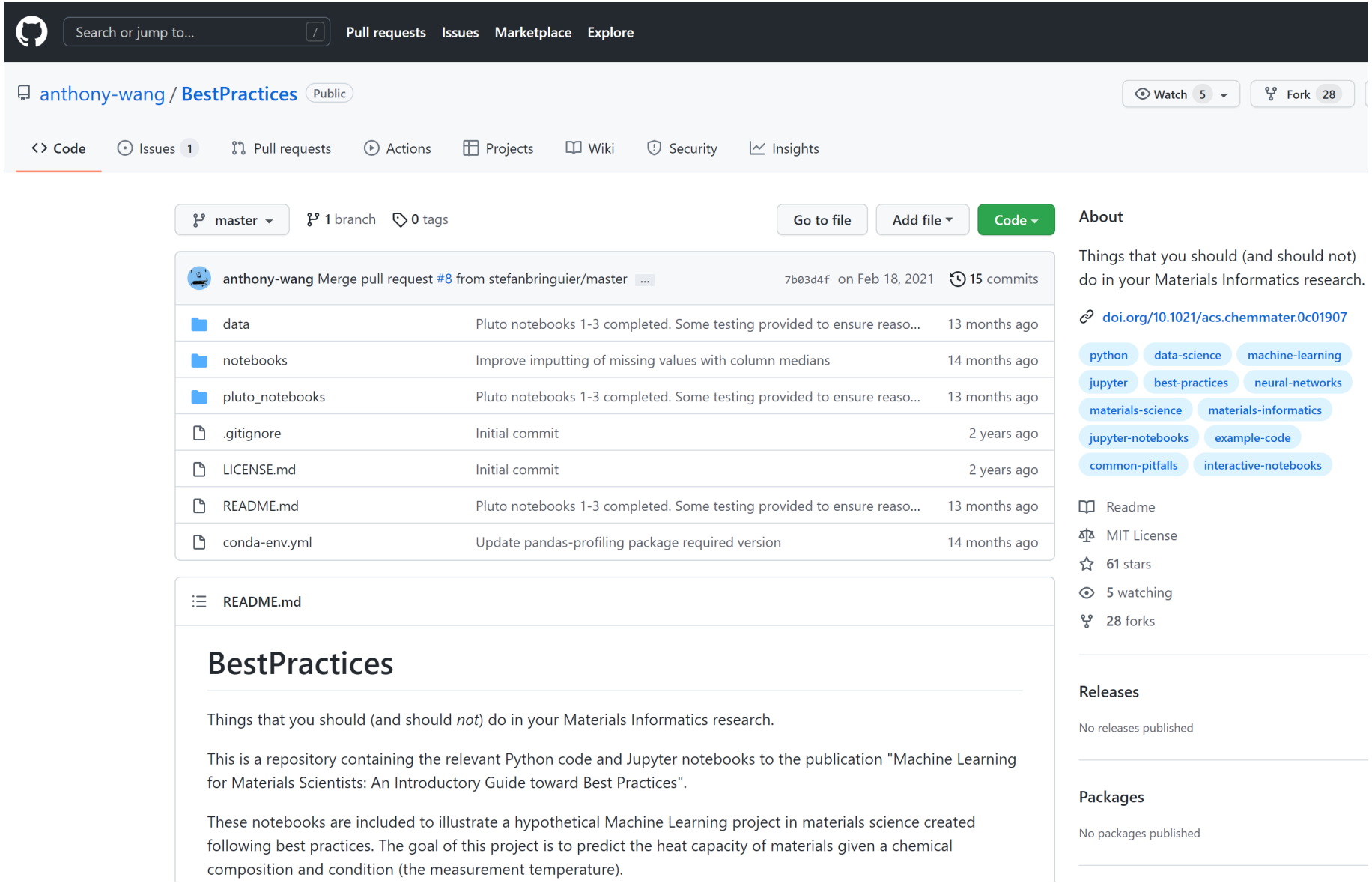
Chemistry of Materials

Abstract

This Methods/Protocols article is intended for materials scientists interested in performing machine learning-centered research. We cover broad guidelines and best practices regarding the obtaining and treatment of data, feature engineering, model training, validation, evaluation and comparison, popular repositories for materials data and benchmarking data sets, model and architecture sharing, and finally publication. In addition, we include interactive Jupyter notebooks with example Python code to demonstrate some of the concepts, workflows, and best practices discussed. Overall, the data-driven methods and machine learning workflows and considerations are presented in a simple way, allowing interested readers to more intelligently guide their machine learning research using the suggested references, best practices, and their own materials domain expertise.



Its companion notebooks are on GitHub



Four things to remember from today

1. Lock the test set first

Choose everything with CV on training data; test once

2. Beat the baseline by more than the noise

Report mean ± spread, in units

3. Looking is fitting

Every tweak made while watching a score fits a little noise

4. Split like you'll use it

Group related rows; new chemistry needs new-chemistry tests

Next session: what did the model learn, and when should you not trust it?

Hands-on: build, tune and grade your own models

45 minutes · Google Colab · no installation needed

Open the Session 3 notebook in Google Colab

1

Go to the course repo

github.com/sp8rks/ACerS_AI_ML_Workshop

2

Click 'Open in Colab'

Session 3: 03_supervised_learning

3

Runtime → Run all

About 6 minutes on a free CPU runtime; read ahead while it runs

4

Edit the 👩‍💻 Your turn cells

report_card() grades every model you build

report_card() reviews your model the way a referee would

Your turn 1: k-nearest neighbors, no scaler, bulk-modulus training set

```
MAE (5-fold CV): 22.5 ± 0.89 GPa   |   train MAE: 18 GPa   |   CV R²: 0.767
✅ Beats predicting the mean (54.3 GPa) by 58%.
✅ Train and CV errors are similar (train/CV = 0.80): not overfitting badly.
💡 Fold-to-fold spread is ±0.89 (4% of the score). Differences between models smaller
than this are probably noise.
⚠️  KNeighborsRegressor depends on feature scale, but there is no scaler. Use
make_pipeline(StandardScaler(), model).
```

Fix what it flags, rerun, and watch the score move

The holdout challenge: one honest look at the test set

Pick a challenge

Thermoelectric power factor, p vs n, zT · moduli · metal vs. non-metal

Baseline + 3 models

report_card on each; act on the feedback

Tune the best

Small search with CV on training data

Write down CV, submit once

Was your estimate honest?

Share: your CV estimate, your one test score, and the feedback that changed what you did